



FORMATION INFORMATIQUE

JavaScript

Les bases pour rendre une page web interactive. Explique simplement, sans blabla inutile.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Salut, c'est quoi JavaScript ?

- [Ce que ce n'est pas](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Exemple pour sentir](#)

2. Chapitre 2 - Ou ecrire du JavaScript ?

- [Ce que ce n'est pas](#)
- [Fichier .js \(le mieux\)](#)
- [Direct dans le HTML / console](#)
- [Mini structure](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Exemple complet](#)
- [En vrai](#)
- [A toi](#)

3. Chapitre 3 - Les variables (des boites a infos)

- [Ce que ce n'est pas](#)
- [Changer et nommer](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Exemple complet](#)
- [En vrai](#)
- [A toi](#)

4. Chapitre 4 - Les types simples

- [Ce que ce n'est pas](#)
- [Assembler du texte](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Exemple complet](#)
- [En vrai](#)
- [A toi](#)

5. Chapitre 5 - Les conditions (if)

- [Ce que ce n'est pas](#)
- [else if, comparaisons, et / ou](#)
- [Petite histoire](#)
- [Erreur classique](#)

- [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)
6. [Chapitre 6 - Les boucles \(repete sans te fatiguer\)](#).
- [Ce que ce n'est pas](#)
 - [while et parcours de liste](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)
7. [Chapitre 7 - Les fonctions \(des recettes reutilisables\)](#).
- [Ce que ce n'est pas](#)
 - [Parametres et return](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)
8. [Chapitre 8 - Les tableaux \(des listes\)](#).
- [Ce que ce n'est pas](#)
 - [Ajouter, retirer, parcourir](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)
9. [Chapitre 9 - Les objets \(des fiches\)](#).
- [Ce que ce n'est pas](#)
 - [Tableau d'objets](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)
10. [Chapitre 10 - Le DOM : trouver des elements](#)
- [Ce que ce n'est pas](#)
 - [Selectionner et verifier](#)
 - [Petite histoire](#)
 - [Erreur classique](#)
 - [Exemple complet](#)
 - [En vrai](#)
 - [A toi](#)

11. Chapitre 11 - Le DOM : changer la page

- [Ce que ce n'est pas](#)
- [Changer le texte](#)
- [Classes, style et creation](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. Chapitre 12 - Les evenements (reagir aux clics)

- [Ce que ce n'est pas](#)
- [Compteur et formulaires](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

13. Chapitre 13 - Mini-projet : compteur de score

- [Ce que ce n'est pas](#)
- [HTML](#)
- [CSS \(simple\)](#)
- [JS](#)
- [Criteres de reussite](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Bonus \(quand la base marche\)](#)
- [En vrai](#)
- [A toi](#)

14. Chapitre 14 - Ce qu'il faut retenir

- [Ce que ce n'est pas](#)
- [Habitudes solides](#)
- [Erreurs classiques a connaitre par coeur](#)
- [Petite histoire](#)
- [Suite possible](#)
- [En vrai](#)
- [A toi](#)

15. Chapitre 17 - Atelier : une todo liste mini

- [Ce que ce n'est pas](#)
- [HTML](#)
- [JS de base](#)
- [Bonus \(quand la base marche\)](#)
- [Petite histoire](#)
- [Erreur a eviter](#)
- [Livrable](#)
- [En vrai](#)

- [A toi](#)

16. [Chapitre 18 - Garder des infos \(localStorage\)](#)

- [Ce que ce n'est pas](#)
- [Nombres et tableaux](#)
- [Brancher sur la todo ou le compteur](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

17. [Chapitre 19 - Erreurs JS frequentes \(et comment les lire\)](#)

- [Ce que ce n'est pas](#)
- [Methode en 5 etapes](#)
- [Erreurs frequentes detaillees](#)
- [Petite histoire](#)
- [Exercice atelier](#)
- [En vrai](#)
- [A toi](#)

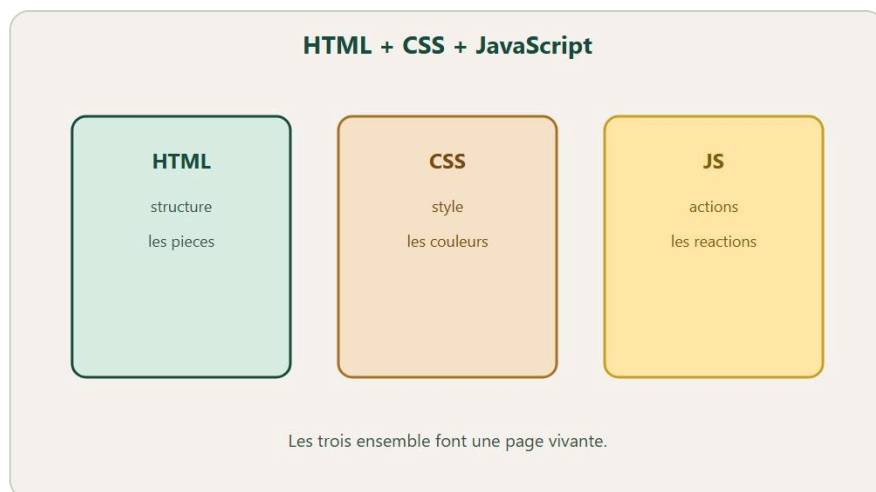
18. [Quiz final - Teste-toi !](#)

- [Questions](#)
- [Reponses](#)
- [Score](#)
- [Petite histoire](#)
- [En vrai](#)
- [A toi](#)

19. [Bravo.](#)

- [Ce que ce n'est pas](#)
- [Ce que tu sais faire maintenant](#)
- [Petite mission](#)
- [Petite histoire](#)
- [La suite quand tu seras pret](#)
- [En vrai](#)
- [A toi](#)

Chapitre 1 - Salut, c'est quoi JavaScript ?



HTML structure. CSS habille. JS fait reagir.

Tu as deja **HTML** (la structure) et **CSS** (le look). JavaScript, c'est le mouvement. La reaction. La vie. Sans JS, une page regarde. Avec JS, elle repond. Tu cliques, elle compte. Tu ecris ton prenom, elle te salue. Tu rates un champ, elle te previent avant l'envoi. Ce n'est pas magie. Ce sont des instructions que le navigateur execute, une apres l'autre, quand quelque chose se passe - ou des le chargement.

Chez DanielCraft, on presente JS comme le cerveau de la page - pas comme un monstre a frameworks, pas comme un club reserve aux "vrais" developpeurs. Le HTML pose le corps : ce qui existe. Le CSS habille : couleurs, style, presentation. **JavaScript** decide, reagit, fait bouger. Lea branche des boutons sur des pages clients. Max a ajoute un compteur de devis sur sa page artisan. Sam montre a ses eleves qu'une page "morte" et une page "vivante", c'est souvent dix lignes de JS clair. Trois metiers, meme logique : petit script, gros effet ressenti.

En 2026, quand quelqu'un dit "j'ai du JavaScript sur mon site", il parle le plus souvent d'interactions simples : menu qui s'ouvre, formulaire qui verifie, compteur qui monte. Derriere, il y a parfois des outils plus gros (React, Vue...). Pour toi, le geste reste le meme : ecrire des instructions claires, tester, corriger. Tu restes le pilote. Le navigateur accelere.

★ A RETENIR

JavaScript = le cerveau de la page. Tu restes le pilote. Lui, il reagit quand tu lui donnes des instructions claires.

Ce que ce n'est pas

Ce n'est pas **Java**. Deux langages differents, malgre le nom proche. Ce n'est pas un remplacement de HTML ou CSS : les trois travaillent ensemble, comme trois metiers sur un chantier. Ce n'est pas un framework (React et cie) des le jour un. Ce n'est pas "installer un truc complique" pour commencer : ca tourne dans le navigateur, souvent avec un simple fichier `.js`. Et ce n'est pas magie : si tu ecris flou, tu obtiens flou.

Ce n'est pas non plus "tout automatiser d'un coup". Commence par un clic qui affiche un message. Monte ensuite vers variables, conditions, DOM. Tu seras pret. Lea rappelle souvent : une premiere victoire vaut mieux qu'un plan de trente pages jamais code.

Ce que tu vas savoir faire

A la fin de ce livre, tu sauras ecrire tes premiers scripts, ranger des infos dans des **variables**, decider avec **if**, repeter avec des boucles, creer des fonctions, manipuler tableaux et objets, trouver et modifier des elements (**DOM**), reagir aux clics, faire un mini compteur, une todo, un peu de localStorage, et lire les erreurs dans la console. Niveau debutant solide. Pas de framework. Juste du JS clair pour comprendre ce que tu touches quand tu ouvres un site.

Niveau debutant solide. Pas besoin d'avoir deja code en Python ou autre. Besoin de curiosite et de tester : JS aide ; il ne remplace pas ta verification ligne par ligne.

Comment lire ce livre

Lis dans l'ordre au debut. Les premiers chapitres posent le sol : ou ecrire, variables, types. Les chapitres du milieu construisent la logique. Les ateliers font faire. Le quiz verifie. Tu peux revenir ensuite a un chapitre precis (DOM, evenements, erreurs) comme a une fiche. A chaque fin, il y a un "A toi". Fais-le. Cinq minutes actives valent mieux qu'une lecture passive de quarante pages.

Chez DanielCraft, on forme des gens qui livrent petit, souvent, proprement - pas des collectionneurs de tutos oublies dans dix onglets. Tu modifies. Tu sauvegardes. Tu rafraichis. Tu regardes la console. Ce rythme bat une soiree de videos sans jamais ouvrir un fichier.



Exemple : un clic, et la page repond (Bonjour / Salut).

Petite histoire

Lea devait rendre un bouton "demander un devis" utile pour un client fleuriste. Sans JS, il menait nulle part - clic, silence. Avec JS, il affiche un message de confirmation, compte les clics de test, et prepare le terrain pour un vrai formulaire plus tard. Quarante minutes, demo nette. Le client comprend. Lea assume le code, pas l'IA.

Max, lui, voulait juste "que ca bouge un peu" sur sa page plomberie. Il a commence par un `alert` basique. Puis un compteur. Puis sa famille a clique en souriant pendant le repas du dimanche. Sam desactive JS dans le navigateur en cours : les eleves voient un site perdre ses superpouvoirs en direct. L'idee rentre sans jargon. Personne ne dit "c'est complique". Ils disent "ah, c'est ca le cerveau".

Erreur classique

Confondre JavaScript et Java. Croire que JS remplace HTML/CSS. Vouloir un framework avant de savoir selectionner un bouton avec `querySelector`. Ou penser "ca ne marche pas" sans ouvrir la `console` (F12). La console est ton ami. On y revient souvent dans ce livre. Autre piege : tout vouloir le premier soir. Un bouton qui repond suffit comme premiere victoire. DanielCraft insiste : petit, clair, testable.

⚠ ATTENTION

Sans console ouverte, tu codes a l'aveugle. F12 des le premier test : tu verras ce que le navigateur te dit vraiment.

En vrai

Ouvre un site que tu visites souvent. Note trois interactions : un menu, un formulaire, un bouton qui change quelque chose. Si tu peux, desactive JavaScript dans les options developpeur de ton navigateur et regarde ce qui casse. Tu verras le "cerveau" disparaître. Puis reactive. Le contraste vaut une heure de theorie. Ensuite, ouvre la console sur n'importe quelle page et tape `console.log("test")`. Si tu vois "test", tu as deja parle a JavaScript. Note en une phrase ce qui "meurt" sans JS : tu poseras mieux tes priorites pour la suite du livre.

A toi

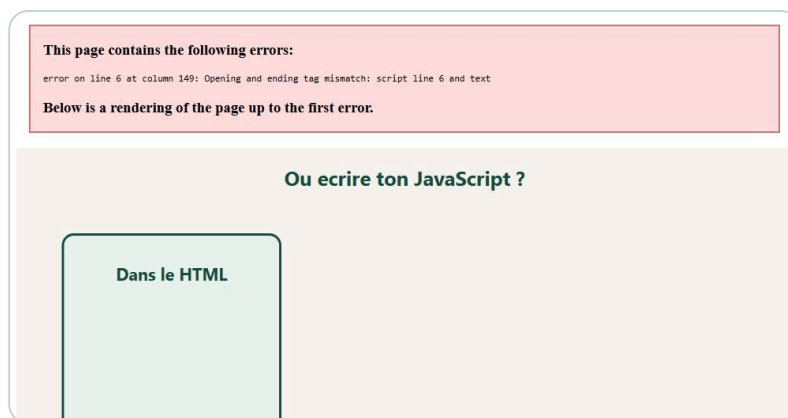
Ecris en trois phrases : (1) une interaction que tu voudrais sur ta page, (2) ce que tu acceptes d'apprendre d'abord (variables, clics...), (3) ce que tu ne feras pas encore (API, framework). Garde ce papier pour le mini-projet du chapitre 13. Chez DanielCraft, ce petit brief vaut plus qu'une heure de tutorials flous : tu sais ou tu vas, et ce que tu refuses pour l'instant.

Exemple pour sentir

```
const bouton = document.querySelector("#direBonjour");
bouton.addEventListener("click", function () {
  alert("Salut ! La page t'a entendu.");
});
```

Tu n'as pas besoin de tout comprendre maintenant. L'idee : JS ecoute et repond. Dans ce livre, on demonte ca piece par piece, avec Lea, Max et Sam comme compagnons de route - et DanielCraft comme fil : petit, clair, testable. Chaque chapitre ajoute une brique. A la fin, la page obeit.

Chapitre 2 - Ou ecrire du JavaScript ?



HTML, fichier .js, ou console : trois endroits pour ecrire.

Tu sais deja ce que fait JavaScript : il reagit, il decide, il fait bouger la page. Maintenant, la question pratique : ou tu ecris ces instructions ? Pas dans le vide. Pas n'importe comment. Trois endroits existent quand tu debutes, et chacun a son role. Le plus important pour un vrai projet : un fichier `.js` separe, branche dans le HTML comme tu branches deja ton CSS. Tu crees `script.js`, tu y mets ton code, tu le relies avec un tag `<script src="script.js">` juste avant `</body>`. Le navigateur charge d'abord la page, puis le script. Les elements HTML existent deja. Ton code peut les trouver sans panique.

Chez DanielCraft, ce reflexe revient dans tous les livres informatique : structure propre, un fichier par role, pas de magma. Lea, freelance web, ne colle presque jamais de gros scripts dans le HTML une fois le projet depasse dix lignes. Max, artisan plombier avec sa page vitrine, a appris apres avoir perdu son compteur de devis dans un tas de balises qu'il n'osait plus toucher. Sam, enseignant, retire des points quand le script est "n'importe ou" dans la copie d'un eleve. Trois metiers, une meme discipline : savoir ou vit le code, et pourquoi.

Tu peux aussi ecrire un petit bloc `<script>` directement dans le HTML pour tester deux minutes. Ou taper dans la **console** du navigateur (F12) pour verifier une idee sans sauvegarder. Les trois servent. La discipline, c'est le fichier `.js` pour tout projet qui dure plus d'une session. La console pour l'exploration rapide. Le script inline pour le tout premier "coucou" qui te donne confiance.

★ A RETENIR

Fichier `.js` separe pour les vrais projets. Console pour tester vite. Script avant `</body>` pour eviter les `null`.

Ce que ce n'est pas

Ce n'est pas "mettre le script n'importe ou et prier". Ce n'est pas encore une lecon sur `defer` ou `async` - tu les croieras plus tard dans des tutos, pas besoin de tout maitriser aujourd'hui. Ce n'est pas une application Node.js ou un serveur : ici, on reste dans le navigateur, sur ta page HTML locale ou en ligne. Et ce n'est pas "ignorer la console" : si tu ne la regardes pas, tu rates la moitie des indices quand quelque chose ne marche pas.

Ce n'est pas non plus copier un script trouve en ligne sans comprendre ou il est branche. Le chemin du fichier, l'emplacement du tag `<script>`, ce sont des details qui font la difference entre "ca marche du premier coup" et "bouton mort depuis trois heures". Lea le dit souvent a ses stagiaires : le branchement, c'est la moitie du travail.

Fichier .js (le mieux)

```
console.log("Salut depuis mon fichier !");
```

Dans le HTML, juste avant `</body>` :

```
<script src="script.js"></script>
```

console.log affiche dans la console. C'est ton ami pour verifier que le script est charge, que ta variable contient ce que tu crois, que ton selecteur a trouve quelque chose. Sans lui, tu codes a l'aveugle. Lea loggue presque tout au debut. Sam exige un log par exercice : si tu ne peux pas montrer ce que tu vois, tu ne peux pas prouver que ca marche.

Le HTML est la scene, le CSS le decor, le fichier JS le livret. Tu le branches en fin de scene pour que les elements existent deja. Si tu lis le livret trop tot, tu cherches un bouton qui n'est pas ne : tu obtiens **null**. Lea resume : "la page d'abord, le cerveau ensuite".

♦ ASTUCE

Place toujours le `<script src="...">` juste avant `</body>` pour debuter. Moins de **null**, moins de panique.

Direct dans le HTML / console

```
<script>
  console.log("Coucou");
</script>
```

Pratique deux minutes pour un test ultra rapide. Sur un vrai projet, prefere le fichier `.js` separe. Et la console : F12, onglet Console, tape `console.log("test")`, Entree. Ideal pour verifier une idee en trente secondes. Max adore ca pour "est-ce que $19 + 3$ donne bien 22 ?". Sam fait tester la console avant le premier fichier.

Mini structure

```
mon-site/
  index.html
  style.css
  script.js
```

Trois fichiers, trois roles. Tu sais ou chercher quand ca casse. Chez DanielCraft, cette arborescence revient dans presque tous les mini-projets. Lea cree ce trio des la premiere heure. Max l'a copie pour sa page artisan. Sam le fait reproduire en classe.

Petite histoire

Lea a passé une heure sur un "bouton mort" pour un client fleuriste : le script était dans le `head`, le bouton plus bas dans le `body`. `querySelector` renvoyait `null` parce que le bouton n'existait pas encore au moment où le script s'exécutait. Elle a déplacé le script avant `</body>`. Marche du premier coup. Quarante minutes perdues pour une ligne déplacée - mais quarante minutes qu'elle n'oubliera plus.

Max avait écrit `scrip.js` au lieu de `script.js` dans le `src`. La console disait 404. Il a lu le message, corrigé une lettre, souri. Sam casse volontairement le chemin en cours pour forcer la lecture d'erreur. Les élèves apprennent à regarder avant de réécrire dix lignes de logique qui n'étaient pas le problème.

Erreur classique

Script trop haut dans le HTML, avant les éléments dont tu as besoin. Résultat : erreur ou `null`. Mauvais chemin de fichier (une lettre suffit : `scrip.js` vs `script.js`). Oublier d'ouvrir la console et croire que "rien ne se passe" alors que le log est là, invisible si tu ne regardes pas. Autre piège : tout coller dans le HTML puis ne plus oser toucher au code par peur de tout casser. Le fichier séparé rassure : un endroit, une vérité, tu peux modifier sans panique.

⚠ ATTENTION

Si tu as `null` sur un élément, vérifie d'abord l'emplacement du script et le chemin du fichier - avant de changer dix lignes de logique.

Exemple complet

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Mon premier JS</title>
</head>
<body>
  <h1 id="titre">Coucou</h1>
  <script src="script.js"></script>
</body>
</html>
```

```
const titre = document.querySelector("#titre");
console.log("Le titre dit :", titre.textContent);
console.log("Script chargé avec succès !");
```

Ouvre la page. Ouvre la console. Tu dois voir les deux messages. Si tu ne vois rien, le chemin ou l'emplacement du script est suspect. Lis l'erreur. Repare. C'est le geste DanielCraft : observer, comprendre, corriger - pas deviner.

En vrai

Cree `index.html` + `script.js`. Relie-les. Affiche ton prenom avec `console.log`. Ouvre la console. Si tu le vois : gagne. Puis casse le chemin volontairement (`scriptt.js`), lis l'erreur 404, repare. Ce geste vaut plus qu'une page de theorie. Lea le fait avec chaque nouveau stagiaire. Max l'a fait une fois, il s'en souvient encore. Toi aussi, tu peux graver ce reflexe aujourd'hui.

A toi

Affiche ton age et ta ville dans la console. Invente un tout petit cas perso (ton jeu prefere, ton animal). Note en une ligne ou tu as place le script et pourquoi. Reflexe DanielCraft : savoir expliquer son branchement, pas juste copier-coller. Si tu ne peux pas expliquer le `src` a voix haute, tu ne le maitrises pas encore - et c'est ok, tu le notes et tu recommences.

Chapitre 3 - Les variables (des boites a infos)



`const` = fixe. `let` = modifiable. Des boites avec un nom.

Une **variable**, c'est une boite avec une etiquette. Tu ranges une valeur dedans. Tu la reutilises plus tard sans recopier partout. Sans variables, tu recopies des nombres et des textes a chaque endroit, et tu te perds des qu'une valeur change. Avec variables, tu changes une fois, tout suit. C'est la base de toute programmation, pas seulement JavaScript. Le jour ou Lea a du changer le nom d'un client dans huit endroits differents parce qu'elle n'avait pas de variable, elle a compris pourquoi ce chapitre existe.

Chez DanielCraft, on enseigne **const** par default et **let** seulement si la valeur doit changer. Lea nomme ses variables comme elle nommerait des dossiers clients : `prenomClient`, `nombreDevis`, pas `x` ou `tmp`. Max a arrete les noms flous apres s'etre embrouille sur un compteur un dimanche soir devant sa famille. Sam refuse les noms incomprehensibles dans les copies : "si tu ne peux pas l'expliquer a voix haute, renomme". Trois metiers, une meme hygiene.

L'etiquette dit `score`, le contenu dit `0`. Tu peux remplacer le contenu d'une boite `let`. Une boite `const` refuse qu'on change le contenu (pour les valeurs simples : tu ne reassigne pas). Si tu ecris sur le mur sans boite (`score = 10` sans `let / const`), le chantier devient sale : d'autres scripts peuvent ecraser ta valeur sans que tu le voies. Max appelle ca "le graffiti du code".

```
let age = 12;
const prenom = "Leo";
```

Avec **let**, tu pourras changer la valeur plus tard. Avec **const**, tu ne reassigne pas : protection volontaire contre les erreurs. Astuce debutant : `const` d'abord. Si tu dois vraiment changer (compteur, score, etat qui evolue), tu passes a `let`. Ce n'est pas une religion. C'est une hygiene qui evite des bugs silencieux quand le projet grandit.

★ A RETENIR

Variable = boite etiquetee. `const` par default, `let` si ca bouge. Nomme clair, declare toujours.

Ce que ce n'est pas

Ce n'est pas `var` (ancien, comportement bizarre, on s'en fiche pour l'instant - si tu le vois dans un vieux tuto, ignore-le). Ce n'est pas une variable sans declaration (`score = 10` tout seul) qui pollue l'espace global et cree des conflits entre scripts. Ce n'est pas "tout en `let` parce que plus flexible" : trop de flexibilité, trop de bugs quand le projet grandit. Et ce n'est pas un objet magique : une variable tient une valeur, point. Tu la nommes. Tu la lis. Tu la changes (si `let`). C'est tout.

Ce n'est pas non plus confondre `=` avec "egal" en maths. En JS, `=` range une valeur dans la boîte. On compare plus tard avec `===` . Lea a perdu une heure la-dessus avant de comprendre que le signe unique ne pose pas la même question que deux signes égal. Tu peux éviter sa douleur en gardant cette distinction en tête des maintenant.

Changer et nommer

```
let score = 0;
score = 10;
score = score + 1; // 11

let nombreDeClics = 0;
const messageBienvenue = "Salut";
```

Oui aux noms clairs : `nombreDeClics` , `messageBienvenue` , `prixTotal` . Non a `x` et `a1` sauf calcul ultra local dans une boucle. Attention : `const ville = "Lyon"; ville = "Paris";` provoque une erreur. C'est voulu. Le navigateur te protège. Tu le remercieras quand tu auras cent lignes de code et que tu ne sauras plus où tu as touché quoi.

♦ ASTUCE

Par défaut, écris `const` . Passe a `let` seulement quand tu dois vraiment réassigner (compteur, score, état qui bouge).

Petite histoire

Lea debuguait un compteur qui "sautait" de façon aléatoire sur une démo client. Deux scripts utilisaient `score` sans `let` / `const` et se marchaient dessus dans l'espace global. Elle a déclaré proprement avec `let score = 0` dans un seul endroit. Calme retrouve. Le client n'a jamais su qu'il y avait eu un mini drame technique derrière son bouton qui comptait enfin correctement.

Max a voulu changer son `const prenom` après une faute de frappe et a rage contre le navigateur pendant cinq minutes. Sam a dit : "il te protège, pas il te punie". Max a sourit le lendemain quand il a compris. Il garde maintenant un post-it : "const sauf compteur". Lea dit : "nomme comme si quelqu'un d'autre devait lire demain matin à 8h sans t'appeler". C'est la règle DanielCraft pour les variables : clarté d'abord, élégance ensuite.

Erreur classique

Oublier `let` ou `const` et écrire `score = 10` tout seul. Réassigner une `const` et croire que le navigateur est "bête". Réutiliser un nom sans le déclarer dans un nouveau fichier. Croire que l'erreur "Assignment to constant variable" est une punition : c'est un garde-fou. Autre piège : noms trop courts (`a` , `tmp2` , `data`) qui ne disent rien quand tu reviens dans deux jours. Lea garde une règle : si tu hésites sur le nom, c'est

que tu n'as pas compris ce que la variable represente.

⚠ ATTENTION

`=` range une valeur dans la variable. Ce n'est pas une comparaison. On compare plus tard avec `===`. Une lettre de difference, un monde de bugs.

Exemple complet

```
const pseudo = "PixelFox";
let niveau = 1;
let xp = 0;

xp = xp + 50;
console.log(pseudo + " a " + xp + " xp");

if (xp >= 50) {
  niveau = niveau + 1;
  xp = xp - 50;
  console.log("Niveau up ! Tu es niveau " + niveau);
}

console.log("Etat final : niveau " + niveau + ", xp " + xp);
```

Lis le code ligne par ligne. `pseudo` ne change jamais : `const`. `niveau` et `xp` evoluent : `let`. La logique du `if` viendra au chapitre 5. Pour l'instant, sens le rythme : declarer, modifier, afficher. C'est le coeur de tout script JavaScript.

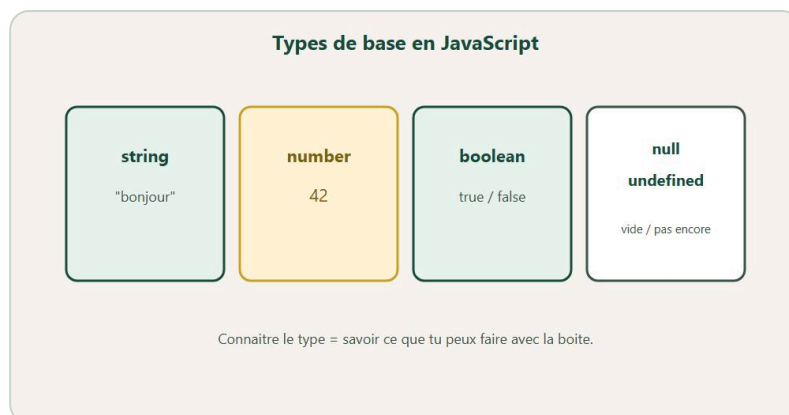
En vrai

Cree `const prenom`, `let points = 0`, ajoute 5 aux points, affiche les deux avec `console.log`. Change seulement les `let`. Laisse les `const` tranquilles. Si la console rale quand tu touches une `const`, tu as compris la protection. Puis essaie de renommer une variable avec une faute de frappe dans un `console.log` : lis l'erreur "is not defined". C'est le navigateur qui te dit : "je ne connais pas cette boite". Lea adore ce moment. Toi aussi, tu peux le provoquer volontairement.

A toi

Trois `const` (prenom, ville, animal) et deux `let` (compteurVisites, derniereNote). Affiche une phrase qui melange tout avec des `+`. Note ta regle perso : "const sauf si...". Chez DanielCraft, cette regle evite des soirees tristes devant l'ecran. Garde-la sur un post-it jusqu'au mini-projet.

Chapitre 4 - Les types simples



string, number, boolean, null, undefined : ce que contient la boîte.

Les valeurs n'ont pas toutes la même forme en JavaScript. Du texte n'est pas un nombre. Un vrai/faux n'est pas une phrase. Si tu mélanges sans faire attention, JavaScript peut coller au lieu de calculer - et tu obtiens des résultats absurdes sans comprendre pourquoi. C'est le piège numéro un des débutants. Chez DanielCraft, on apprend les **types** tôt pour éviter le classique `"19" + 3` qui donne `"193"` au lieu de `22`. Une fois que tu vois ça une fois, tu ne l'oublies plus.

Lea le croise dans les formulaires web : tout ce qui sort d'un champ est du texte, même si l'utilisateur tape des chiffres. Max l'a vu sur un "total devis" qui affichait "150030" au lieu de 1530 parce qu'il additionnait deux strings. Sam le fait tester en classe jusqu'au "ah !" collectif quand `"5" + 2` s'affiche à côté de `5 + 2`. Trois métiers, une même leçon : regarde la forme de la valeur avant de calculer.

Une **string**, c'est du texte entre guillemets. Un **number**, c'est un nombre (entier ou decimal). Un **boolean**, c'est `true` ou `false`. `undefined` veut dire "pas encore défini". `null` veut dire "volontairement vide". Tu n'as pas à en faire des tartines maintenant : sache que ça existe, et regarde les guillemets avant de blâmer les maths. `"5"` est une brique texte. `5` est une brique nombre. Les coller avec `+` ne donne pas le même résultat.

```
const phrase = "Bonjour";
const prix = 19.99;
const quantite = 3;
const total = prix * quantite;
const estConnecte = true;
```

Ce que ce n'est pas

Ce n'est pas encore les objets et tableaux (chapitres 8 et 9). Ce n'est pas "les guillemets sont optionnels pour le texte" : sans guillemets, JavaScript cherche une variable qui s'appelle comme ton mot. Ce n'est pas croire que `==` arrangera tout : on préférera `===` pour comparer sans surprise de type. Et ce n'est pas paniquer sur `typeof` : c'est un outil pour vérifier, pas un examen. Tu l'utilises pour debug, pas pour impressionner.

Ce n'est pas non plus penser que le navigateur "devine" ce que tu veux. Il suit des règles. `"5" + 2` colle du texte. `5 + 2` calcule. Point. Lea dit : "le navigateur n'est pas bête. Il fait ce que tu lui demandes. Parfois tu lui demandes n'importe quoi."

★ A RETENIR

String = texte. Number = nombre. Boolean = vrai/faux. Guillemets = texte. Pas de guillemets sur un nombre = calcul.

Assembler du texte

```
const prenom = "Maya";
console.log("Salut " + prenom);
console.log(`Salut ${prenom}`);
```

Les **backticks** (accent grave, touche a cote du 1) permettent d'insérer `${...}` directement dans la phrase. Plus moderne, tres pratique pour les messages longs. Une fois que tu y goutes, tu y reviens souvent. Lea prefere les backticks pour les mails clients. Max les utilise pour ses recus de devis. Sam montre les deux syntaxes puis laisse les eleves choisir leur preferee.

Quand tu tapes dans un formulaire, le navigateur te donne des briques texte. Toi, tu decides si tu calcules (convertir en number avec `Number(...)`) ou si tu affiches tel quel (string). `"19" + 3` donne `"193"` (collage). `19 + 3` donne `22` (calcul). Si la valeur vient d'un champ, convertis avant les maths.

Petite histoire

Max additionnait un prix tape dans un champ (texte `"150"`) avec une quantite (nombre `3`). Resultat : `"1503"` affiche fierement sur sa page plomberie. Lea lui a montre `Number(prixChamp)` avant le calcul. Facture correcte : 450. Max a note "Number avant calcul" sur son carnet. Sam projette `"5" + 2` et `5 + 2` cote a cote : les eleves votent a main levee, puis verifient dans la console. Le type devient concret. Plus personne ne dit "le navigateur est bete". On dit "j'ai colle du texte".

Erreur classique

```
const total = "19" + 3; // "193" - collage
const totalOk = 19 + 3; // 22 - calcul
```

Ou convertir avec `Number("19")` si ca vient d'un champ. Autre piege : melanger simples et doubles guillemets sans coherence dans un fichier (pas grave, mais choisis un style). Et oublier que `typeof null` dit `"object"` - bizarre, historique, on le note sans en faire un drame. Ce n'est pas toi qui as mal code. C'est JS qui traine une vieille bizarrerie.

⚠ ATTENTION

Tout ce qui sort d'un champ de formulaire est du texte (string), meme si ca "ressemble" a un nombre. Convertis avant de calculer.

Exemple complet

```
const client = "Maya";
const article = "Casque";
const prixUnitaire = 29.99;
const quantite = 2;
const tauxTva = 0.2;

const sousTotal = prixUnitaire * quantite;
const montantTva = sousTotal * tauxTva;
const totalTTC = sousTotal + montantTva;

const recu = `
Facture pour ${client}
- ${quantite} x ${article} : ${sousTotal.toFixed(2)} EUR
- TVA : ${montantTva.toFixed(2)} EUR
= Total : ${totalTTC.toFixed(2)} EUR
`;
console.log(recu);
```

`.toFixed(2)` arrondit a deux decimales pour l'affichage. Pratique pour les prix. Lea l'utilise sur tous ses devis demo. Max aussi sur sa page artisan. Tu verras ce genre de detail partout ou l'argent apparait dans le code.

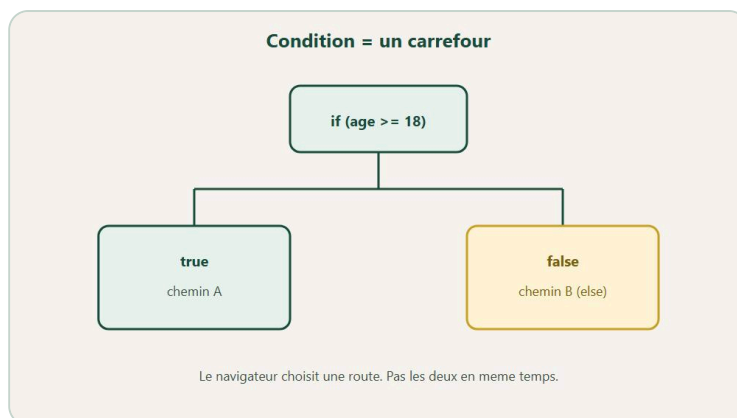
En vrai

Fais un mini ticket de caisse : prix, quantite, total, message avec backticks. Affiche `typeof` sur chaque variable dans la console. Observe les etiquettes : `"string"`, `"number"`, `"boolean"`. Tu verras les formes des briques. Puis teste `"5" + 2` et `5 + 2` cote a cote. Note la difference en une phrase. Ce geste vaut une demi-heure de cours. Max l'a note sur un post-it. Toi aussi, tu peux.

A toi

Teste `"5" + 2` et `5 + 2`. Note la difference en une phrase claire. Cree une string, un number, un boolean. Assemble un message avec backticks et `${}`. Tu poses une brique solide pour les conditions du chapitre suivant. Chez DanielCraft, ce chapitre evite des heures de debug absurde sur des totaux qui collent au lieu de calculer.

Chapitre 5 - Les conditions (if)



if / else : le navigateur choisit un chemin.

Parfois tu veux : si ca, alors ca. Sinon, autre chose. Les **conditions** sont le volant de ton script. Sans elles, le programme fait toujours la meme chose, bete et droit, quelle que soit la situation. Avec elles, tu decides : majeur ou mineur, mot de passe correct ou refuse, canicule ou froid. C'est la ou JavaScript commence a "penser" - pas comme un humain, mais en suivant des regles que tu ecris. Chez DanielCraft, on insiste sur `===` pour comparer, et on reserve `=` a l'affectation. Lea a perdu une heure sur un `=` dans un `if`. Max aussi. Sam le piege volontairement en cours - et tout le monde tombe une fois. Puis plus jamais.

Chaque `if` est une question. Chaque reponse ouvre ou ferme une porte. Si tu ecris mal la question (`=` au lieu de `===`), tu ne compares pas : tu forces une reponse. Lea appelle ca "soudoyer le portier sans le vouloir". Max a compris le jour ou son acces chantier s'ouvrait pour tout le monde. Sam fait jouer "portier" a voix haute avant le code : la logique se clarifie avant qu'une ligne soit ecrite.

```

const age = 15;
if (age >= 18) {
  console.log("Majeur");
} else {
  console.log("Mineur");
}
  
```

Tu peux enchaîner avec `else if`. Tu combines avec `&&` (et) et `||` (ou). Tu gardes les tests lisibles : une idee claire par branche. Si ta foret d'if devient illisible, decoupe - ou passe par une fonction. Le code lisible, c'est du code que tu peux relire dans six mois sans te demander ce que tu voulais dire.

★ A RETENIR

Dans un `if`, `===` compare. `=` assigne. Une lettre, un monde de bugs. Dis la regle a voix haute avant de coder.

Ce que ce n'est pas

Ce n'est pas un `switch` obligatoire (utile plus tard pour beaucoup de cas identiques). Ce n'est pas `==` "parce que plus court" : `===` protege des surprises de types melanges. Ce n'est pas une foret de `if` imbriques sur vingt niveaux - personne ne peut lire ca, toi inclus dans deux jours. Et ce n'est pas "mettre un `=` dans le `if` pour aller plus vite". Ce geste assigne. Il ne compare pas. Resultat : toujours "vrai",

toujours le mauvais chemin.

Lea rappelle : un `if` bien écrit, c'est une phrase en français traduite en code. Si tu ne peux pas la dire à voix haute, le code sera flou aussi.

else if, comparaisons, et / ou

```
const note = 14;
if (note >= 16) {
  console.log("Excellent");
} else if (note >= 10) {
  console.log("C'est valide");
} else {
  console.log("On revise");
}

const aUnTicket = true;
const estVip = false;
if (aUnTicket || estVip) {
  console.log("Tu peux entrer");
}
```

Comparaisons utiles : `===`, `!==`, `>`, `<`, `>=`, `<=`. Tu les verras partout. Apprends-les comme des outils de poche. Le `&&` veut dire "les deux doivent être vrais". Le `||` veut dire "au moins un des deux suffit". Simple. Puissant.

⚠ ATTENTION

Dans un `if`, écris toujours `===` (compare). Jamais un seul `=` (assigne). Une lettre, un monde de bugs.

Petite histoire

Lea validait un mot de passe avec `if (motDePasse = "secret123")`. Toujours "OK". Effroi. Puis compréhension : elle assignait au lieu de comparer. Max a fait un accès chantier : âge + invitation. Il a mis un `=` par réflexe. Sam fait jouer "portier" à voix haute avant le code : "si âge `>=` 18 ou invitation, alors entrer". Quand tu peux expliquer la décision à voix haute, le code suit. Les élèves qui passent par la voix haute font moins d'erreurs. Ce n'est pas magique. C'est de la traduction.

Erreur classique

```
if (motDePasse = "secret123") { /* assigne ! */ }
if (motDePasse === "secret123") { /* compare */ }
```

Oublier les accolades sur un `if` d'une ligne puis ajouter une deuxième ligne "dedans" qui n'y est pas. Lis toujours le bloc. Autre piège : trop de conditions dans une seule ligne - découpe pour rester lisible. Max a appris à écrire ses conditions sur plusieurs lignes quand ça dépasse deux tests. Lea découpe en fonctions dès qu'elle voit plus de trois `else if`. Avant d'écrire le `if`, dis la règle à voix haute. Puis code. Moins d'erreurs.

Exemple complet

```
const age = 16;
const aInvitation = true;
const estBlackliste = false;

function peutEntrer(age, invitation, blackliste) {
  if (blackliste) return "Acces refuse : compte bloque";
  if (age < 13) return "Acces refuse : trop jeune";
  if (age >= 18 || invitation) return "Bienvenue !";
  return "Acces refuse : invitation requise";
}

console.log(peutEntrer(age, aInvitation, estBlackliste));
```

Lis chaque branche comme une phrase. Blackliste ? Refuse. Trop jeune ? Refuse. Majeur ou invite ? Bienvenue. Sinon ? Invitation requise. C'est lisible. C'est testable. C'est du DanielCraft : clair, pas malin pour rien.

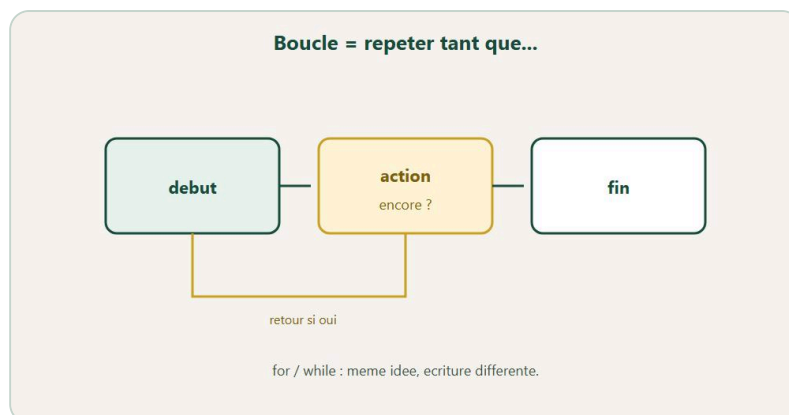
En vrai

Cree une variable `motDePasse` en dur. Si elle vaut `"secret123"`, affiche "OK", sinon "Refuse". Change la valeur. Observe. Puis casse volontairement avec un seul `=` dans le `if` et vois le piege : tout passe, toujours. Remets `===`. Le contraste enseigne mieux qu'un paragraphe de theorie. Lea fait ce test avec chaque stagiaire. Max s'en souvient encore.

A toi

Variable `temperature` : `>= 35` canicule, `>= 25` chaud, `>= 15` correct, sinon froid. Change plusieurs fois et observe chaque branche. Ecris en une phrase ta regle d'or sur `===`. Dis-la a voix haute avant le prochain `if` que tu ecriras. Chez DanielCraft, cette regle evite des soirees tristes devant l'ecran.

Chapitre 6 - Les boucles (repete sans te fatiguer)



Repete une action tant que la condition est vraie.

Une **boucle**, c'est faire la meme chose plusieurs fois sans copier-coller quarante lignes identiques. Afficher une table de multiplication. Parcourir une liste de prenom. Compter de 1 a 100. Repeter tant qu'une condition reste vraie. Chez DanielCraft, on aime les boucles parce qu'elles revelent tout de suite si tu comprends le compteur - ou si tu crees une boucle infinie qui freeze le navigateur. Lea les utilise pour generer des listes de tags sur les pages clients. Max pour numeroter des etapes sur sa page devis. Sam pour les tables de multiplication en cours. Trois usages, une meme logique : le compteur doit avancer.

Une boucle, c'est une machine. Tu regles le debut, la fin, le pas. Ou tu dis "continue tant qu'il reste des vies". Si tu n'enlèves jamais de vie, la machine ne s'arrete pas. Lea appelle ca "oublier le frein". Max a vu Chrome geler une fois sur sa page plomberie. Il n'oublie plus d'incrémenter. Sam chronometre en cours : "qui trouve l'oubli ?". Les eleves deviennent des chasseurs de freins.

```
for (let i = 1; i <= 5; i = i + 1) {
  console.log("Tour numero " + i);
}
```

Lecture : commence a 1, tant que `i <= 5`, a chaque tour ajoute 1. Le **while** repete tant qu'une condition reste vraie. Attention : si tu oublies de faire evoluer la condition, boucle infinie. Le navigateur rale. Toi aussi. Coupe l'onglet, respire, ajoute le frein. Max l'a vecu une fois. Il n'oublie plus.

★ A RETENIR

Boucle = repeter avec un compteur qui avance. Oublie le frein = page freeze. Nombre de tours connu -> for . "Tant que" -> while .

Ce que ce n'est pas

Ce n'est pas encore `map` / `filter` (plus tard, quand les tableaux seront familiers). Ce n'est pas "while partout" : si tu connais le nombre de tours, **for** est souvent plus clair. Ce n'est pas une excuse pour ne pas penser : une boucle mal bornée freeze la page et te fait perdre ton travail non sauvegarde. Et ce n'est pas "copier-coller dix `console.log`" : la boucle existe precisement pour eviter ca.

Sam dit aux eleves : une boucle, c'est une machine. Si tu ne regles pas le frein, la machine ne s'arrete pas. Ce n'est pas une metaphore jolie. C'est la realite du code.

while et parcours de liste

```
let vies = 3;
while (vies > 0) {
  console.log("Il reste " + vies + " vies");
  vies = vies - 1;
}

const fruits = ["pomme", "banane", "kiwi"];
for (const fruit of fruits) {
  console.log(fruit);
}
```

`for...of` parcourt un tableau simplement. On creuse les tableaux au chapitre 8. Pour debuter, c'est souvent le plus lisible : tu dis "pour chaque fruit, affiche-le". Pas besoin de gerer l'index a la main.

⚠ ATTENTION

Dans un `while`, verifie toujours que quelque chose change a chaque tour. Sinon : boucle infinie, page freeze.

Petite histoire

Max a freeze Chrome avec un `while` ou il affichait `i` sans l'incrémenter. Il a coupe l'onglet, un peu pale, puis a compris. Lea montre toujours ou le compteur avance avant de lancer la boucle. Sam chronometre : "qui trouve l'oubli en moins de trente secondes ?". Les eleves deviennent des chasseurs de freins. L'erreur devient un reflexe a chercher, pas une humiliation devant la classe.

Lea, elle, utilise les boucles pour generer des menus de navigation sur les sites clients : dix liens, une boucle, pas dix lignes copiees. Max numerote ses etapes de devis. Sam fait calculer la somme de 1 a 100 en classe. Quand quelqu'un obtient 5050, la boucle devient concrete.

Erreur classique

```
let i = 0;
while (i < 5) {
  console.log(i);
  // oubli de i = i + 1
}
```

Corrige en faisant evoluer `i`. Autre piege : commencer a 1 ou a 0 selon le besoin, et se tromper sur la borne `<=` vs `<`. Affiche le compteur. Compte a la main une fois. Ca clarifie. Lea trace la boucle sur papier avant de coder quand c'est un peu complique. Si tu hesites entre `for` et `while` : nombre de tours connu -> `for`. "Tant que" une condition -> `while`.

Exemple complet

```
const mots = ["le", "chat", "dort", "sur", "tapis"];
let compteur = 0;
for (const mot of mots) {
  if (mot.length >= 4) {
    compteur = compteur + 1;
    console.log(mot + " compte !");
  }
}
console.log("Total mots longs : " + compteur);

for (let n = 1; n <= 10; n = n + 1) {
  console.log("3 x " + n + " = " + (3 * n));
}
```

Tu melanges boucle, condition et compteur. C'est le trio qui reviendra partout dans le livre. Lis ligne par ligne. Sens le rythme. Puis modifie un nombre et relance.

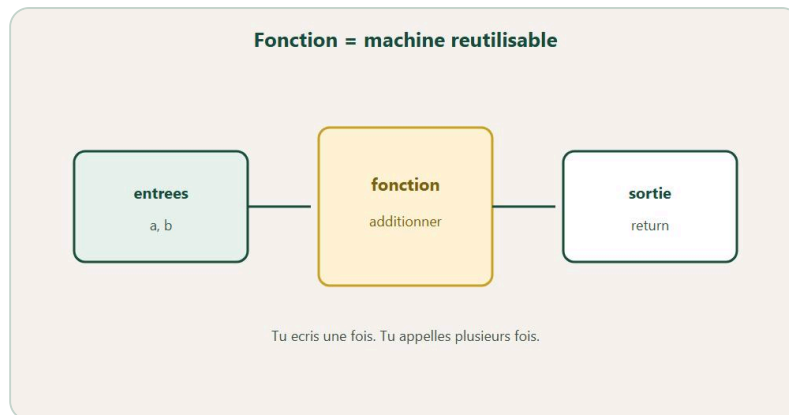
En vrai

Affiche la table de 7 (7x1 a 7x10) avec un `for`. Puis les pairs de 2 a 20. Deux boucles, deux victoires. Si tu freeze, cherche le frein avant de tout recommencer : ou le compteur devrait avancer ? Ajoute-le. Relance. Tu as appris plus qu'avec une page de theorie. Sam chronometre ce geste en classe. Max l'a appris une fois pour toutes.

A toi

Calcule la somme de 1 a 100 (tu dois obtenir 5050). Parcours un tableau de prenom avec "Salut, ...". Bonus : arrete si tu trouves `"stop"`. Ecris ta phrase anti-boucle-infinie sur un post-it. Reflexe DanielCraft : toujours montrer ou le compteur avance, avant de lancer la boucle. Tu prepares les tableaux du chapitre suivant avec un sol solide.

Chapitre 7 - Les fonctions (des recettes réutilisables)



Entrees -> fonction -> sortie. Ecrire une fois, appeler souvent.

Une **fonction**, c'est une recette. Tu la définis une fois. Tu l'appelles quand tu veux. Tu copies moins. Tu corriges à un seul endroit. Ton code devient plus clair parce que chaque morceau a un nom qui dit ce qu'il fait. Chez DanielCraft, on préfère plusieurs petites fonctions à une usine de soixante lignes que personne n'ose toucher. Lea découpe "calculer", "afficher", "valider". Max a compris le jour où il a corrigé un total de devis à un seul endroit au lieu de trois. Sam refuse les "fonctions fourre-tout" dans les copies : une fonction, une mission.

Les paramètres sont les ingrédients. Le **return**, c'est le gâteau sorti du four. `console.log` dans la cuisine, ce n'est pas la même chose que donner le gâteau à table. Lea dit : "log pour toi, return pour le programme". Sam projette les deux versions côte à côte en cours. Le "ah" collectif quand `undefined` apparaît sans `return`. Ensuite, plus personne ne confond afficher et renvoyer.

```
function direSalut() {
  console.log("Salut !");
}
direSalut();
direSalut();
```

Avec des **paramètres** (ingrédients) : `direSalutA("Nora")`. Avec un résultat : **return**. Sans `return`, tu récupères souvent `undefined` si tu attendais une valeur. Ce n'est pas un bug du navigateur. C'est toi qui as oublié de rendre le gâteau. Lea le répète : log pour toi, return pour le programme.

★ A RETENIR

Ecrire une fonction ne suffit pas : il faut l'appeler. Et si tu as besoin du résultat, il faut un `return`.

Ce que ce n'est pas

Ce n'est pas obligatoire d'écrire des flèches `=>` tout de suite : **function** suffit largement pour débuter. Ce n'est pas "une fonction = tout le programme" : découpe. Ce n'est pas oublier d'appeler : une fonction écrite mais jamais appelée ne fait rien, comme une recette jamais cuisinée. Et ce n'est pas `return` optionnel quand tu as besoin du résultat. `console.log` dans la cuisine, ce n'est pas la même chose que donner le gâteau à table.

Max a mis six mois a vraiment sentir la difference entre afficher et renvoyer. Toi, tu peux la sentir ce chapitre avec un petit test volontaire.

Parametres et return

```
function direSalutA(prenom) {  
  console.log("Salut " + prenom);  
}  
  
function double(n) {  
  return n * 2;  
}  
const resultat = double(4); // 8
```

Petite forme moderne a reconnaitre : `const triple = (n) => n * 3;`. Pour l'instant, `function` suffit largement. Tu croiseras les fleches plus tard sans panique. Lea les utilise parfois pour des callbacks courts. Max reste sur `function` pour l'instant. Sam montre les deux sans imposer.

Petite histoire

Lea avait copie le meme calcul de moyenne trois fois dans un script client. Un bug, trois corrections. Elle a fait `moyenne(a, b, c)`. Un fix. Le client n'a jamais su qu'il y avait eu un mini drame derriere la facture qui s'affichait enfin correctement.

Max ecrivait des fonctions sans `return` et affichait `undefined` en croyant a un bug du navigateur. Sam projette les deux versions cote a cote. Le "ah" collectif. Ensuite, plus personne ne confond afficher et renvoyer. C'est un de ces moments ou une ligne de code change ta facon de lire le reste du livre.

Erreur classique

```
function carre(n) {  
  n * n; // resultat perdu  
}  
function carreOk(n) {  
  return n * n;  
}
```

Oublier d'appeler. Donner trop de responsabilites a une seule fonction. Nommer `fonction1`. Autre piege : appeler avec `()` trop tot dans un `addEventListener` (on y revient au chapitre evenements). Lea garde une regle : si ta fonction fait plus de quinze lignes, decoupe. `moyenne(12, 16)` renvoie `14`. Tu ranges le resultat dans une variable, puis tu l'affiches. Calcul et affichage restent separes.

⚠ ATTENTION

Sans `return`, tu recuperes `undefined` si tu attendais une valeur. Ce n'est pas le navigateur qui est bete : c'est toi qui as oublie de rendre le resultat.

Exemple complet

```
function moyenne(a, b, c) {
  return (a + b + c) / 3;
}
function mention(note) {
  if (note >= 16) return "Tres bien";
  if (note >= 14) return "Bien";
  if (note >= 10) return "Passable";
  return "Insuffisant";
}
function afficherBulletin(prenom, n1, n2, n3) {
  const moy = moyenne(n1, n2, n3);
  console.log(prenom + " : " + moy.toFixed(1) + "/20 - " + mention(moy));
}
afficherBulletin("Leo", 12, 15, 18);
```

Trois fonctions, trois roles. `moyenne` calcule. `mention` decide. `afficherBulletin` assemble. C'est le modele DanielCraft : petit, clair, testable. Tu peux tester `moyenne` seule sans toucher au reste.

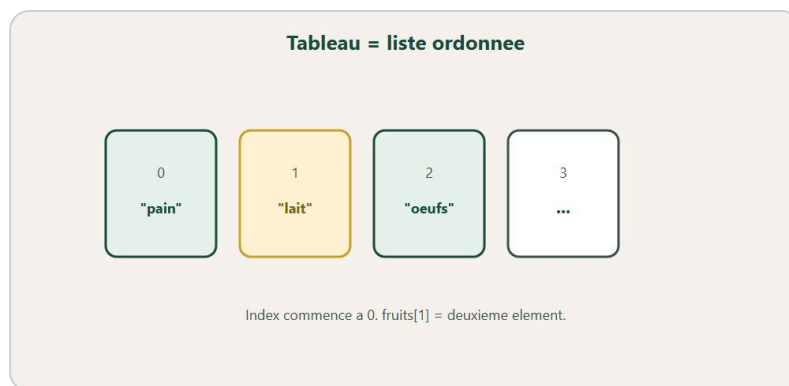
En vrai

Ecris `moyenne(a, b)` pour 12 et 16. Verifie le resultat dans la console. Puis retire le `return` une seconde : observe `undefined`. Remets. Le contraste enseigne mieux qu'un paragraphe de theorie. Lea le fait systematiquement. Max a mis six mois a vraiment sentir la difference : toi, tu la sens ce chapitre.

A toi

Ecris `aireRectangle(largeur, hauteur)` avec `return`. `estMajeur(age)` qui renvoie un boolean. `saluer(prenom)` qui affiche sans `return`. Appelle les trois. Note en deux phrases la difference `return` vs `log`. Competence pilote pour la suite du livre : sans ca, tu confondras encore affichage et resultat. Chez DanielCraft, on separe les roles des le debut.

Chapitre 8 - Les tableaux (des listes)



Liste ordonnee. Index commence a 0.

Un **tableau**, c'est une file d'elements ranges dans un ordre. Tu y mets des courses, des jeux, des taches, des scores, des noms de clients. Tu ajoutes, tu retires, tu parcoures. L'**index** commence a 0. Oui, a 0. C'est bizarre au debut, puis ca devient un reflexe. Sans tableaux, tu te retrouves avec dix variables `fruit1`, `fruit2`, `fruit3` et tu pleures des qu'il faut en ajouter un quatrieme. Avec un tableau, tu as une seule boite qui grandit.

Chez DanielCraft, le tableau est la structure "liste" par default avant les objets. Lea, freelance web, les utilise pour menus de navigation et listes de tags sur les pages clients. Max, artisan plombier, range ses fournitures du mois dans un tableau pour les afficher proprement. Sam, enseignant, fait compter a partir de zero jusqu'a ce que ca rentre dans la tete des eleves. Trois metiers, une meme idee : une liste ordonnee, un index qui part de zero, une longueur qui dit combien il y a d'elements.

Pense a des casiers numerotes 0, 1, 2. Le casier 0 tient le premier element. Si tu demandes le casier 3 alors qu'il n'y en a que trois (0, 1, 2), tu obtiens `undefined`. Quand tu fais `push`, tu ouvres un nouveau casier a la fin. Quand tu fais `pop`, tu retires ce qui est dans le dernier. La longueur change. L'ordre compte.

```
const courses = ["pain", "lait", "oeufs"];
console.log(courses[0]); // pain
courses[1] = "lait d'avoine";
console.log(courses.length); // 3
```

`push` ajoute a la fin. `pop` retire le dernier. `includes` verifie la presence. Tu parcoures avec `for` ou `for...of`. Une liste qui vit : c'est ca, le tableau. Tu n'as pas besoin de tout savoir d'un coup. Tu as besoin de sentir que la liste grandit et retrecit sous tes doigts.

★ A RETENIR

Tableau = liste ordonnee. Index 0 pour le premier. Dernier index = `length - 1`. Jamais `length` comme index.

Ce que ce n'est pas

Ce n'est pas un objet (fiches nommees : chapitre suivant). Ce n'est pas "l'index 1 est le premier" - le premier est 0. Ce n'est pas `length` egal au dernier index : le dernier index est `length - 1`. Si tu ecris `courses[courses.length]`, tu obtiens `undefined`. Et ce n'est pas encore les methodes avancees `map` / `filter`. On reste sur le solide : creer, lire, ajouter, retirer, parcourir.

Ce n'est pas non plus "tout mettre dans un tableau geant sans reflechir". Une liste de fruits, oui. Melanger scores, images et mots de passe dans le meme tableau sans structure, non. Lea dit : une liste, un genre d'elements. Sam ajoute : si tu ne peux pas expliquer ce que contient la liste en une phrase, elle est trop floue.

Ajouter, retirer, parcourir

```
courses.push("beurre");
const dernier = courses.pop();

for (let i = 0; i < courses.length; i = i + 1) {
  console.log(courses[i]);
}
for (const item of courses) {
  console.log(item);
}
if (courses.includes("pain")) {
  console.log("On a du pain");
}
```

Le `for` classique te donne l'index : utile si tu as besoin du numero. Le `for...of` te donne directement l'element : plus lisible quand tu n'as pas besoin de l'index. `includes` repond vrai ou faux. Trois outils, trois situations. Lea prefere `for...of` pour afficher. Max utilise souvent l'index quand il numerote une facture. Sam montre les deux et laisse choisir.

♦ ASTUCE

Pour le premier element : `[0]`. Pour le dernier : `[tableau.length - 1]`. Jamais `[tableau.length]` - ca donne `undefined`.

Petite histoire

Max cherchait le "dernier" avec `courses[courses.length]` et voyait `undefined` sur sa page de fournitures. Il a rage cinq minutes. Lea a dit `length - 1`. Il a note sur un post-it. Depuis, il ne se trompe plus. Sam fait une course en classe : "qui donne l'index de banane dans `['pomme', 'banane', 'kiwi']` ?" La moitie leve la main pour "2". Puis on verifie. Puis on refait. Compter a partir de zero devient un reflexe, pas une blague de developpeur.

Lea, elle, utilise les tableaux pour generer des menus : dix liens, une boucle, pas dix lignes copiees. Max numerote ses etapes de devis. Sam fait afficher une liste de prenomes avec "Salut, ..." en boucle. Quand la liste grandit d'un `push` et que l'affichage suit, le tableau devient concret.

Erreur classique

Confondre index et longueur. Modifier une liste en bouclant d'une façon qui saute des éléments (plus tard, quand tu filtreras en place). Croire que `push` renvoie le tableau (il renvoie la nouvelle longueur). Pour débiter, affiche le tableau après chaque action avec `console.log`. Tu verras évoluer la liste. Ça rassure. Autre piège : commencer à compter à 1 "parce que c'est plus naturel" et tout décaler d'une case. Lea dit : accepte le zéro. C'est le dialecte du langage. `["a", "b"]` : index de `"a"` = 0, `length` = 2, dernier index = 1. Trois nombres, trois pièges classiques.

Exemple complet

```
const courses = ["pain", "lait", "oeufs"];
courses.push("fromage");
for (let i = 0; i < courses.length; i = i + 1) {
  console.log(i + 1 + ". " + courses[i]);
}
if (courses.includes("lait")) {
  console.log("lait est dans la liste");
}
const retire = courses.pop();
console.log("Retire : " + retire);
console.log("Reste :", courses);
```

Lis ligne par ligne. Tu ajoutes, tu numérotés à l'affichage (1, 2, 3... pour les humains), tu vérifies une présence, tu retires. C'est le rythme d'une todo, d'un panier, d'un inventaire. Chez DanielCraft, on aime ce genre d'exemple parce qu'il se montre en trente secondes.

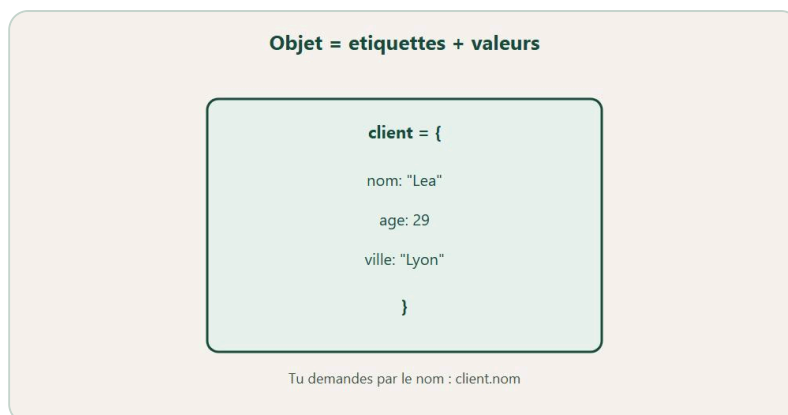
En vrai

Crée une liste de quatre jeux. Fais un `push` d'un cinquième. Affiche tous avec une boucle. Observe `length` avant et après. Puis retire le dernier avec `pop` et réaffiche. Tu dois sentir la liste respirer. Si `undefined` apparaît, regarde ton index. Corrige. Relance. Cinq minutes actives valent mieux qu'une page de théorie.

A toi

Cinq films. Affiche le premier et le dernier. Fais un `push`, un `pop`, un parcours `for...of` avec `.toUpperCase()` sur chaque titre. Observe la liste évoluer dans la console. Note en une phrase ta règle anti-`undefined` sur le dernier élément. Tu prépares la todo de l'atelier : sans tableau solide, la liste de tâches ne tiendra pas.

Chapitre 9 - Les objets (des fiches)



Étiquettes + valeurs. Tu demandes par le nom.

Un **objet**, c'est une fiche avec des cases nommées. Prenom, score, actif. Titre, pages, lu. Prix, quantité, client. Au lieu d'une liste anonyme ou tout est "case 0, case 1", tu as des étiquettes claires. Tu lis `joueur.prenom`. Tu changes `joueur.score`. Tu comprends sans deviner. Chez DanielCraft, on dit : tableau pour lister, objet pour décrire une chose. Souvent, tu combines : un **tableau d'objets**. C'est le modèle le plus fréquent des qu'une page web devient un peu riche.

Lea, freelance web, modèle des clients ainsi : nom, email, projet, statut. Max, artisan, représente un devis : client, montant, date, payé. Sam, enseignant, représente un élève : prenom, moyenne, présent. Trois métiers, une même structure. Une chose = une fiche. Une liste de choses = un tableau de fiches. Une fois que tu sens ça, beaucoup de tutoriels deviennent lisibles d'un coup.

Pense à une fiche cartonnée : nom, prenom, classe. Chaque case a un nom. Tu ne cherches pas "la case numéro 2" : tu cherches "prenom". Un tableau de fiches, c'est le classeur. Max a arrêté ses "deux tableaux parallèles" (prenoms d'un côté, scores de l'autre) le jour où il a vu la fiche unique. Sam montre les deux modèles et demande lequel survit à l'ajout d'une propriété `ville`. L'objet gagne.

```

const joueur = {
  prenom: "Sam",
  score: 120,
  estActif: true
};
console.log(joueur.prenom);
joueur.score = joueur.score + 10;

```

Tu lis avec `joueur.prenom` ou `joueur["prenom"]`. Si la clé a un espace, les crochets deviennent nécessaires. Le point marche pour les noms simples. Les crochets sauvent les cas bizarres. Tu n'as pas à tout mémoriser. Tu as à savoir où regarder quand ça rale.

★ A RETENIR

Tableau = liste. Objet = fiche. Tableau d'objets = classeur. Trois outils, trois jobs.

Ce que ce n'est pas

Ce n'est pas un tableau (même si les deux se croisent constamment). Ce n'est pas encore du JSON fichier sur disque (proche, mais on reste en JavaScript vivant). Ce n'est pas "tout mettre dans un seul objet géant du monde" avec cinquante propriétés mêlées. Des petites fiches claires. Une chose = un objet. Une liste de choses = un tableau. Et ce n'est pas confondre `joueur.score` avec une variable libre `score` flottant quelque part : la propriété vit dans l'objet.

Ce n'est pas non plus "plus compliqué qu'un tableau". C'est souvent plus clair. Lea dit aux stagiaires : si tu as besoin de nommer les cases, prends un objet. Si tu as besoin d'un ordre et d'une longueur, prends un tableau. Si tu as les deux besoins, combine.

Tableau d'objets

```
const equipe = [
  { prenom: "Sam", score: 120 },
  { prenom: "Lea", score: 95 }
];
console.log(equipe[0].prenom);
equipe[1].score = equipe[1].score + 5;
```

Quand utiliser quoi ? Tableau : plusieurs éléments du même genre. Objet : une chose avec plusieurs infos. Combo : liste de choses riches. C'est le modèle le plus fréquent sur le web débutant. Une liste de produits. Une liste de tâches. Une liste de joueurs. Chaque élément est une fiche. Tu parcoures avec `for...of`. Tu lis `item.nom`. Tu modifies `item.fini`. Ça tient.

Si tu te retrouves avec `prenoms[]` et `scores[]` qui doivent rester alignés : passe à un tableau d'objets. Moins d'index qui dérivent, moins de bugs silencieux.

Petite histoire

Max stockait prénom et score dans deux tableaux parallèles et se trompait d'index des qu'il triait ou supprimait. Lea a dit : un objet par joueur. Fin du chaos. En une après-midi, sa mini page de scores est devenue lisible. Sam montre les deux modèles et demande lequel survit à l'ajout d'une propriété `ville`. L'objet gagne à chaque fois. Les élèves refactorisent. Ils ne veulent plus revenir en arrière.

Lea, elle, livre des listes de témoignages clients en tableau d'objets : texte, auteur, note. Max un devis multi-lignes : désignation, quantité, prix. Sam un bulletin : matière, note, coefficient. Trois scènes, une structure. Chez DanielCraft, on répète : nomme les cases, range les fiches.

Erreur classique

```
const user = { "nom complet": "Leo Martin" };
console.log(user["nom complet"]); // ok
// user.nom complet -> erreur de syntaxe
```

Oublier la virgule entre propriétés. Confondre `joueur.score` et `joueur[score]` sans guillemets (JavaScript cherche alors une variable `score`). Autre piège : vouloir un tableau là où une fiche suffit, ou l'inverse. Ou créer un objet géant "app" avec tout dedans et ne plus oser le toucher. Préfère des petites fiches. Assemble-les dans un tableau si besoin.

⚠ ATTENTION

`joueur["score"]` avec guillemets lit la propriété. `joueur[score]` sans guillemets lit la variable `score` comme cle. Une paire de guillemets, un monde de bugs.

Exemple complet

```
const inventaire = [
  { nom: "Zelda", heures: 40, fini: true },
  { nom: "Minecraft", heures: 120, fini: false },
  { nom: "Tetris", heures: 5, fini: true }
];
function afficherJeux(jeux) {
  for (const jeu of jeux) {
    const statut = jeu.fini ? "fini" : "en cours";
    console.log(jeu.nom + " (" + jeu.heures + "h) - " + statut);
  }
}
afficherJeux(inventaire);
```

Tu melanges objet, tableau, boucle et fonction. C'est le combo qui revient partout ensuite. Lis ligne par ligne. Sens le rythme. Puis ajoute un jeu avec `push` et relance. Chez DanielCraft, ce genre de mini inventaire se montre en une minute et convainc mieux qu'un discours.

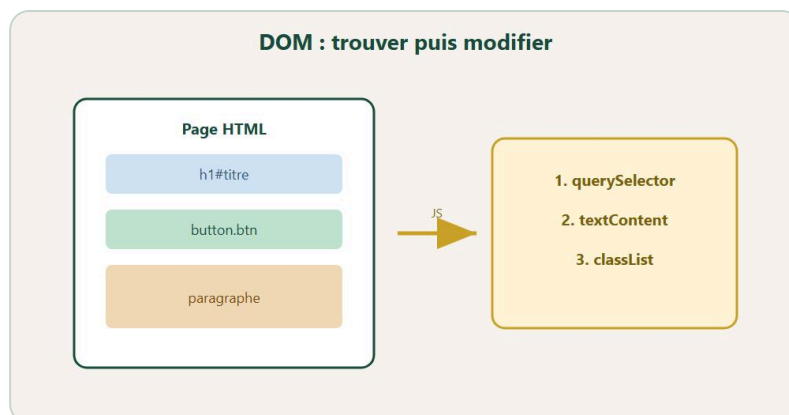
En vrai

Crée un objet `livre` : titre, pages, `lu` (boolean). Affiche une phrase complète avec backticks. Change `lu` à `true`. Reaffiche. Puis transforme-le en tableau de deux livres et parcours-les. Tu dois sentir la différence entre "une fiche" et "un classeur". Si tu bloques sur le point ou les crochets, loggue l'objet entier avec `console.log(livre)`.

A toi

Objet `moi` avec prenom, age, ville, et `hobbies` (un tableau dedans). Affiche "J'aime : ..." en parcourant les hobbies. Puis un tableau de deux amis-objets avec les memes cles. Affiche chaque ami. Tu melanges objet et tableau : c'est le reflexe a garder pour la suite du livre, pour la todo enrichie, et pour les projets DanielCraft.

Chapitre 10 - Le DOM : trouver des elements



On trouve un element, puis on le modifie.

Le **DOM**, c'est la page vue par JavaScript. Des elements. Ce que tu as ecrit en HTML, accessible en JS. Avant de changer quoi que ce soit, tu dois trouver. Sans trouver, tu modifies le vide. Tu obtiens `null`. Tu rages. Puis tu comprends. Chez DanielCraft, on compare `querySelector` aux selecteurs CSS : `#id`, `.classe`, `balise`. Si tu as fait le livre HTML/CSS, tu es deja a moitie chez toi. Lea vit la-dedans tous les jours. Max a sourit le jour ou `#titre` a marche du premier coup. Sam dit : si tu as `null`, tu n'as pas encore trouve - arrete-toi avant de modifier.

En 2026, quand quelqu'un dit "je selectionne un element", il parle le plus souvent de `querySelector`. Derriere, il y a d'autres methodes (`getElementById`, `getElementsByClassName` ...). Pour toi, une seule suffit largement pour debuter. Le geste reste le meme : ecrire le bon selecteur, verifier avec `console.log`, continuer seulement si tu tiens quelque chose. Tu restes le pilote. Le DOM te repond.

Tu es dans un magasin. Le DOM est le plan. `querySelector` est "allez chercher l'article avec cette etiquette". Si l'etiquette n'existe pas, tu reviens les mains vides (`null`). Ensuite seulement tu peux etiqueter, deplacer, changer le prix. Lea dit : "trouver avant de modifier" - comme une loi d'atelier. Max compare ca a chercher une piece dans sa camionnette : si tu n'as pas la bonne reference, tu ne demontes pas le robinet au hasard.

```
<h1 id="titre">Salut</h1>
<button class="btn">Clique</button>
```

```
const titre = document.querySelector("#titre");
const bouton = document.querySelector(".btn");
console.log(titre);
console.log(bouton);
```

`querySelectorAll` renvoie une liste. `console.log(titre)` te dit si tu tiens quelque chose. `null` = selecteur faux, ou script trop tot. Rappel du chapitre 2 : script juste avant `</body>`. Sans ca, tu cherches un acteur qui n'est pas encore entre en scene. Lea le repete aux stagiaires jusqu'a ce que ca rentre. Max l'a appris a la dure. Sam le piege volontairement en cours.

★ A RETENIR

Trouver avant de modifier. `querySelector` + `console.log`. Si `null`, corrige le selecteur ou l'emplacement du script.

Ce que ce n'est pas

Ce n'est pas encore modifier (chapitre suivant). Ici, tu cherches et tu verifies. Ce n'est pas jQuery. Ce n'est pas "getElementById obligatoire" : `querySelector` suffit largement pour debuter. Et ce n'est pas ignorer `null` : tu verifies. Toujours. Avant de toucher `.textContent` ou d'ecouter un clic. Modifier `null`, c'est l'erreur classique "Cannot read properties of null".

Ce n'est pas non plus confondre `#` et `.`. `#titre` cible un id. `.titre` cible une classe. Une lettre, un monde. Lea a perdu vingt minutes sur une majuscule dans un id. Max aussi. Sam fait un jeu : trois mauvais selecteurs, un bon. Les eleves `console.log` jusqu'a trouver.

Selectionner et verifier

```
const items = document.querySelectorAll("li");
console.log(items.length);
console.log(titre);
console.log(titre.textContent);
```

Meme logique qu'en CSS. C'est pour ca que ca devient vite naturel si tu as fait le livre HTML/CSS. `#` pour un id, `.` pour une classe, `balise` pour un type d'element. Une lettre, un monde. `querySelector` prend le premier match. `querySelectorAll` prend tous les matches. Pour debuter, loggue toujours. Si tu vois l'element dans la console, tu tiens. Si tu vois `null`, tu cherches encore.

♦ ASTUCE

Des que tu selectionnes, fais un `console.log`. Si tu vois `null`, arrete-toi : corrige le selecteur ou l'emplacement du script avant de continuer.

Petite histoire

Lea a eu un bug "impossible" sur une page fleuriste : id `Titre` en HTML, `#titre` en JS. Casse. Une majuscule. Vingt minutes. Max avait le script dans le `head`. `null` systematiquement. Il a deplace avant `</body>`. Marche. Sam fait un jeu en classe : trois mauvais selecteurs, un bon. Les eleves `console.log` jusqu'a trouver. Le geste devient un reflexe, pas une punition. Personne ne dit "JS est casse". On dit "je n'ai pas encore trouve".

Lea rappelle aussi : si tu as plusieurs `.btn`, `querySelector` ne prend que le premier. Pour tous, `querySelectorAll`. Max a clique sur le "mauvais" bouton pendant une semaine avant de comprendre. Sam le montre cote a cote. Le contraste enseigne.

Erreur classique

Selecteur faux. Script trop tot. Confondre `#` et `.`. Oublier que `querySelector` ne prend que le premier match. Pour plusieurs, `querySelectorAll`. Autre piege : croire que "ca ne marche pas" sans jamais logger ce que tu as vraiment dans les mains. Ou corriger dix lignes de logique alors que le probleme etait une faute dans l'id HTML. Chez DanielCraft, on lit d'abord ce que la console montre. Ensuite on agit.

⚠ ATTENTION

`#titre` et `.titre` ne sont pas la meme chose. Id vs classe. Verifie le HTML avant de blamer JS.

Exemple complet

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Selection DOM</title>
</head>
<body>
  <h1 id="titre">Ma page</h1>
  <button class="btn">Action</button>
  <ul>
    <li>Un</li>
    <li>Deux</li>
  </ul>
  <script src="script.js"></script>
</body>
</html>
```

```
const titre = document.querySelector("#titre");
const bouton = document.querySelector(".btn");
const items = document.querySelectorAll("li");

console.log(titre.textContent);
console.log(bouton);
console.log("Nombre de li :", items.length);
```

Ouvre la page. Ouvre la console. Tu dois voir le texte du titre, l'element bouton, et 2. Si tu vois `null`, deplace le script ou corrige le selecteur. C'est le geste DanielCraft : observer, comprendre, corriger.

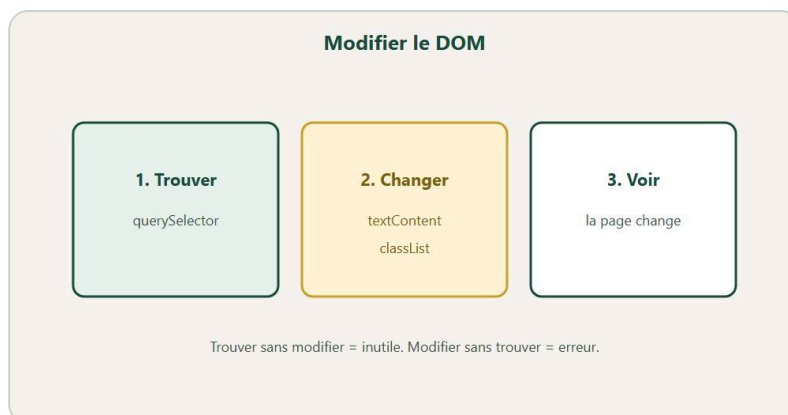
En vrai

Cree une page avec un `h1` id. Selectionne-le. `console.log` son `textContent`. Si `null`, deplace le script avant `</body>`. Puis casse volontairement le selecteur (`#Titre` au lieu de `#titre`). Lis. Repare. Ce contraste vaut une demi-heure de cours. Lea le fait avec chaque stagiaire. Max s'en souvient encore.

A toi

Selectionne un bouton par classe et tous les `li`. Affiche les longueurs et les textes. Invente un cas perso (ton score, ton jeu, ta page artisan). Note ton selecteur gagnant sur un post-it. Chez DanielCraft, trouver avant de modifier est une loi d'atelier - tu la reutiliseras au chapitre suivant des que tu changeras la page.

Chapitre 11 - Le DOM : changer la page



Trouver, changer, voir le resultat a l'ecran.

Tu as appris a trouver un element avec `querySelector`. Maintenant tu vas le modifier. Changer le texte d'un titre. Ajouter une classe qui allume une couleur. Creer un paragraphe qui n'existait pas une seconde avant. La page n'est plus figee comme une photo : ton script la sculpte en direct, dans le navigateur, en memoire. Ce n'est pas magie. Ce sont des methodes JavaScript qui agissent sur l'arbre HTML deja charge.

Chez DanielCraft, on prefere `textContent` pour debuter plutot que `innerHTML`. Pourquoi ? Parce que `textContent` met du texte, point. Pas de balises interpretees, pas de surprise si quelqu'un tape du code bizarre dans un champ. On prefere aussi les **classes CSS** plutot que d'eparpiller vingt lignes de `style`. en JavaScript. JS decide l'etat (actif, ouvert, erreur). CSS decide l'apparence. Lea toggle des classes `.active` sur des cartes clients. Max change un message de statut sur sa page plomberie. Sam montre la difference entre `textContent` et `innerHTML` avec un exemple prudent devant ses eleves. Trois metiers, meme reflexe : petit changement cible, bien place.

Tu as deja trouve la pancarte (chapitre precedent). Maintenant tu changes le texte dessus, tu colles un sticker colore (une classe), ou tu ajoutes une nouvelle pancarte avec `createElement` puis `appendChild`. Le magasin evolue sans reconstruire le batiment entier. Lea appelle ca "chirurgie locale". Max prefere ca aux "tout casser et recommencer". Sam dit : "le DOM, c'est la maquette vivante de ta page - pas le fichier sur le disque, la copie en cours dans le navigateur".

★ A RETENIR

Trouve d'abord, modifie ensuite. `textContent` pour du texte simple. `classList` pour l'etat. `createElement` + `appendChild` pour ajouter.

Ce que ce n'est pas

Ce n'est pas encore les evenements clic - on y va juste apres. Ici, tu changes souvent au chargement de la page, pas en reaction a un geste utilisateur. Ce n'est pas reecrire tout le `body` pour changer un mot : c'est chirurgie locale, pas demolition totale. Ce n'est pas `innerHTML` avec des donnees non fiables venant de l'utilisateur : la, tu ouvres la porte aux bugs et aux failles. Et ce n'est pas styler vingt proprietes en JS si une seule classe CSS suffit. JS decide. CSS habille. Ne melange pas les roles.

Ce n'est pas non plus "modifier le fichier HTML sur disque". Quand tu changes le DOM en JavaScript, tu changes la page en memoire. Au prochain refresh, tu repars du fichier original. C'est normal. C'est le fonctionnement du web. Lea le rappelle souvent aux clients qui s'attendent a ce que le script "ecrive dans le HTML" comme dans Word.

Changer le texte

```
const titre = document.querySelector("#titre");
titre.textContent = "Nouveau titre";
```

`textContent` remplace tout le texte visible de l'element. Simple. Sur. Pour debuter, c'est ton outil numero un. `innerHTML` peut interpreter des balises HTML : utile si tu sais ce que tu fais, mais plus risque si le contenu vient de l'utilisateur ou d'une source externe. Lea a deja vu un client injecter du HTML casse parce qu'il croyait que "innerHTML = plus fort". Non. Plus puissant, pas plus sur.

Classes, style et creation

```
const carte = document.querySelector(".carte");
carte.classList.add("active");
carte.classList.remove("active");
carte.classList.toggle("active");

titre.style.color = "teal"; // ok pour tester
// Sur un vrai projet, prefere souvent les classes

const p = document.createElement("p");
p.textContent = "Je viens d'apparaître";
document.body.appendChild(p);
```

Le couple `classList` + CSS est elegant : tu ajoutes `.active` en JS, tu definis `.active { background: gold; }` en CSS. Tu changes toute la charte sans retoucher dix scripts. `createElement` cree l'element en memoire. `appendChild` l'accroche au document pour qu'il devienne visible. Sans `appendChild`, tu as un element fantome : il existe en JS, mais personne ne le voit. Pour un etat (actif, ouvert, erreur), cree une classe CSS et toggle-la en JS. Moins de `style.` eparpilles, plus facile a maintenir.

⚠ ATTENTION

`createElement` seul ne suffit pas. Il faut `appendChild` (ou un equivalent) pour que l'element apparaisse dans la page.

Petite histoire

Lea remplaçait des blocs entiers en `innerHTML` pour mettre a jour une liste de temoignages. Chaque fois, les ecouteurs d'evenements qu'elle avait branches sur les boutons "lire plus" disparaissaient. Elle est passee a des updates cibles : changer le texte d'un `p`, toggle une classe sur un `div`. Probleme resolu en une session. Max mettait dix proprietes `style.` en ligne sur son bandeau de statut. Sam lui a dit : "une classe `.highlight`, c'est tout". Au prochain changement de charte graphique, Max a gagne une heure. Trois lecons, un meme reflexe : petit changement visible, bien place, testable tout de suite.

Erreur classique

Modifier sans vérifier que l'élément existe : si `querySelector` renvoie `null`, tu plantes. Utiliser `innerHTML` pour du texte simple : c'est comme utiliser un marteau pour visser. Oublier `appendChild` après `createElement` : l'élément reste invisible. Croire que changer le JS change le fichier HTML sur disque : non, ça change la page en mémoire. Au refresh, tu repars du fichier. Autre piège : mettre le script avant que le HTML existe. On y revient souvent. Script avant `</body>`, sélecteur correct, élément présent.

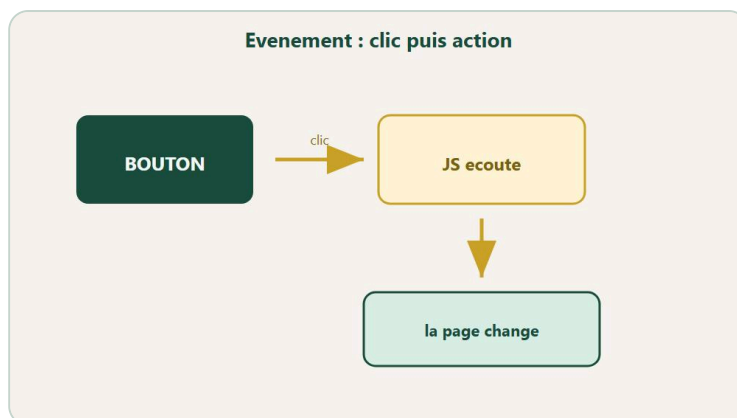
En vrai

Ouvre une page avec un paragraphe et un titre. Change le texte du paragraphe au chargement avec `textContent`. Ajoute une classe `highlight` à un titre (avec le CSS correspondant dans ta feuille de style). Crée un `p` dynamiquement avec ton prénom et ajoute-le au body. Tu dois voir les trois changements sans cliquer encore. Si ça marche, tu as le geste. Le chapitre suivant branchera l'interaction. Si quelque chose manque, loggue l'élément : `null` ? Corrige avant de paniquer.

A toi

Crée un `p` dynamiquement avec ton prénom et ajoute-le au body. Toggle une classe sur un bouton au chargement (même sans clic encore : tu peux toggle au load pour tester). Vérifie que le CSS de la classe existe vraiment. Prépare le terrain pour les événements. Style DanielCraft : petit changement visible, puis on branche l'interaction au chapitre suivant. Note en une ligne ce que tu as changé à l'écran.

Chapitre 12 - Les evenements (reagir aux clics)



Clic -> JS ecoute -> la page change.

Jusqu'ici, ton JavaScript agissait surtout au chargement. La page se chargeait, le script modifiait quelque chose, fin de l'histoire. Maintenant, la page devient vraiment interactive. Tu écoutes. Tu réponds. Un clic, un message. Un clic, un compteur qui monte. Un submit de formulaire, une vérification avant envoi. C'est ici que le web cesse d'être une brochure et devient un outil. Chez DanielCraft, `addEventListener` est le geste clé : "quand cet événement arrive sur cet élément, fais ça".

Lea branche des boutons CTA sur des pages clients : "demander un devis", "voir les tarifs". Max compte les clics de test sur son bouton "Appeler maintenant" pour montrer à sa femme que la page "vit". Sam fait applaudir la classe au premier compteur qui marche en direct - c'est un moment. Trois métiers, même logique : le HTML propose l'élément, le JS dispose de la réaction quand on écoute. Sans listener, la sonnette existe mais personne ne réagit.

Une sonnette à la porte. L'événement, c'est l'appui. Le listener, c'est toi derrière la porte, prêt à réagir. Sans listener, la sonnette existe (HTML) mais personne ne bouge. Avec listener, la maison s'anime : message, compteur, validation. Lea dit : "le HTML propose, le JS dispose - mais seulement quand on écoute". Commence par `click` : le plus clair, le plus testable. Une interaction nette vaut dix démos floues.

★ A RETENIR

`addEventListener` = "quand cet événement arrive sur cet élément, fais ça". Tu donnes la recette, tu ne la lances pas tout de suite.

```
<button id="bouton">Clique-moi</button>
<p id="message"></p>
```

```
const bouton = document.querySelector("#bouton");
const message = document.querySelector("#message");
bouton.addEventListener("click", function () {
  message.textContent = "Bravo, tu as cliqué !";
});
```


Ce que ce n'est pas

Ce n'est pas `onclick="..."` colle partout dans le HTML. Ca marche, oui, mais on prefere separer : HTML pour la structure, JS pour le comportement. Ce n'est pas ecouter avant d'avoir l'element en memoire : si le script est trop haut ou l'id faux, tu ecoutes le vide. Ce n'est pas oublier `preventDefault` sur un submit si tu geres toi-meme le formulaire : sans ca, la page recharge et tes logs disparaissent. Et ce n'est pas dix ecouteurs identiques copies-colles sans fonction : une logique claire, un listener propre, eventuellement une fonction reutilisable.

Ce n'est pas non plus appeler la fonction tout de suite dans `addEventListener`. Tu passes la recette, tu ne lances pas le plat. `addEventListener("click", maFonction)` - pas `maFonction()`. Lea a deja vu un junior se demander pourquoi son alert s'affichait au chargement au lieu du clic. La reponse etait dans les parentheses.

Compteur et formulaires

```
let clics = 0;
bouton.addEventListener("click", function () {
  clics = clics + 1;
  message.textContent = "Clics : " + clics;
});

form.addEventListener("submit", function (event) {
  event.preventDefault();
  // ton code ici
});
```

Tu peux garder un compteur avec `let clics = 0` et l'afficher a chaque clic. Tu croiseras aussi `input` (ecriture dans un champ), `submit` (envoi de formulaire), parfois `mouseover` ou `keydown`. Commence par `click`. `preventDefault` dit au navigateur : "laisse, je gere". Utile pour ne pas recharger la page pendant tes tests. Sans ca, tu cliques, la page part, tu ne comprends rien. Lea a perdu une heure la-dessus un mardi matin. Max aussi. Maintenant ils le mettent en premier sur tout formulaire test.

♦ ASTUCE

Premier test d'un listener : mets `console.log("clic")` dedans. Preuve de vie avant la logique metier. Si tu vois "clic" en console, le fil est branche.



Exemple : Clique-moi, et le message apparaît.

Petite histoire

Lea avait un formulaire de contact qui "mangeait" ses logs a chaque test : la page rechargeait, les `console.log` disparaissaient, elle croyait que son code ne marchait pas. `preventDefault` a tout change en une ligne. Max mettait le listener sur un id faux (`#bouton` au lieu de `#btn`) : silence total, pas d'erreur visible si tu ne log pas. Sam exige un `console.log("clic")` en premier dans chaque handler avant d'ecrire le vrai code. Preuve de vie. Ensuite seulement la logique metier. Trois scenes, une methode : ecoutier le bon element, prouver que ca reagit, puis construire.

Erreur classique

Element `null`, donc erreur au `addEventListener` : selecteur faux ou script trop haut. Oublier que la fonction listener n'est pas appelee avec `()` dans le add : tu passes la fonction, tu ne l'executes pas tout de suite. Mettre la logique hors du listener et s'etonner que ca ne reagit pas au clic : le code s'execute une fois au chargement, pas a chaque clic. Croire que "ca marche" sans jamais cliquer vraiment : teste. Clique dix fois. Souris. C'est vivant ou ce ne l'est pas.

⚠ ATTENTION

Ecris `addEventListener("click", maFonction)` - pas `maFonction()`. Tu donnes la recette, tu ne la lances pas tout de suite.

En vrai

Cree un bouton et un paragraphe. Branche un compteur : chaque clic ajoute 1, affiche le total dans le `p`. Clique dix fois. Regarde le chiffre monter. Puis ajoute un mini formulaire avec `preventDefault` qui affiche le prenom saisi dans un paragraphe sans recharger la page. Si les deux marchent, tu as le coeur du web interactif. Le mini-projet du chapitre 13 va assembler ca proprement. Si le formulaire recharge, cherche `preventDefault` avant de recrire dix lignes.

A toi

Ajoute un bouton reset qui remet le compteur a zero. Puis un mini formulaire avec `preventDefault` qui affiche le prenom saisi dans un `p`. Teste chaque piece separement avant d'assembler. Tu prepares le mini-projet compteur. Facons DanielCraft : petit, clair, testable, un geste a la fois. Note en une phrase ce qui a bloque, s'il y a eu un blocage.

Chapitre 13 - Mini-projet : compteur de score



HTML + JS + etat + DOM : le compteur en quatre briques.

On assemble. C'est le moment ou tout ce que tu as appris se rencontre : **variables**, **DOM**, **evenements**. Le but est simple et concret : afficher un score, le faire monter avec un bouton +1, le faire descendre avec -1 (sans passer sous zero), le remettre a zero avec reset. Simple en apparence. Mais ca assemble la chaine complete que tu retrouveras sur presque chaque page interactive. Chez DanielCraft, ce mini-projet est le "hello world" utile : tu touches, ca repond, tu comprends le fil du debut a la fin.

Lea utilise des compteurs similaires pour des demos clients : "regardez, votre page peut compter les demandes de devis en test". Max a fait le sien pour un mini jeu avec ses neveux un dimanche pluvieux. Sam note : si les trois boutons marchent, le score ne descend jamais sous zero, et le code est range avec une fonction **afficher**, c'est reussi. Pas besoin de perfection. Pas besoin de localStorage encore. Besoin d'un geste fini que tu peux montrer sans rougir.

Un tableau de score au basketball de salon. Tu ajoutes un point, tu retires un point (sans passer en negatif), tu remets a zero pour une nouvelle manche. L'afficheur au mur, c'est le DOM (**#score**). Les boutons sur la table, ce sont les evenements. La variable **score** en memoire, c'est le vrai chiffre. Trois pieces, un systeme. Change la memoire, puis mets a jour l'ecran. Dans cet ordre. Toujours.

Ce que ce n'est pas

Ce n'est pas un jeu AAA avec des graphismes et du son obligatoire. Ce n'est pas localStorage encore - ca viendra dans l'atelier todo et le chapitre 18. Ce n'est pas "parfait ou rien" : une version 1 qui marche bat une version mentale jamais codee. Et ce n'est pas copier-coller sans retaper : les doigts apprennent. Sam interdit parfois le copier-coller en atelier. Lea chronometre parfois : "version 1 en vingt minutes". L'objectif, c'est finir.

HTML

```
<h1>Score</h1>
<p id="score">0</p>
<button id="plus">+1</button>
<button id="moins">-1</button>
<button id="reset">Reset</button>
<script src="script.js"></script>
```

Place le script avant `</body>`. Les ids doivent correspondre exactement a ce que tu selectionnes en JS. Une faute de frappe, et tu ecoutes le vide.

CSS (simple)

```
body {
  font-family: Georgia, serif;
  text-align: center;
  margin-top: 3rem;
  background: #f4f1ec;
}
#score { font-size: 3rem; }
button {
  font: inherit;
  margin: 0.3rem;
  padding: 0.5rem 0.9rem;
}
```

Le CSS n'est pas l'objet du chapitre, mais un minimum de style rend la demo presentable. Max a ajoute un fond bleu pale. Ses neveux ont trouve ca "officiel".

JS

```
const scoreEl = document.querySelector("#score");
const plusBtn = document.querySelector("#plus");
const moinsBtn = document.querySelector("#moins");
const resetBtn = document.querySelector("#reset");
let score = 0;

function afficher() {
  scoreEl.textContent = score;
}

plusBtn.addEventListener("click", function () {
  score = score + 1;
  afficher();
});
moinsBtn.addEventListener("click", function () {
  if (score > 0) {
    score = score - 1;
    afficher();
  }
});
resetBtn.addEventListener("click", function () {
  score = 0;
  afficher();
});
afficher();
```

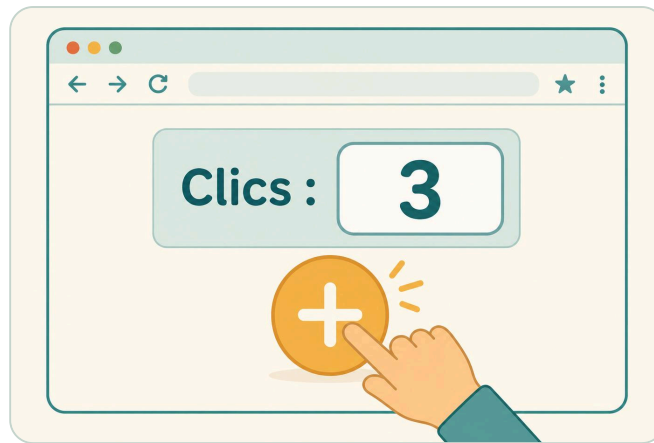
La fonction **afficher** centralise la mise a jour de l'ecran. Chaque listener change `score`, puis appelle `afficher()`. Pas de duplication. Lea insiste : un seul endroit qui ecrit dans le DOM pour le score. Si demain tu changes l'affichage, tu touches une fonction.

★ A RETENIR

Change la memoire (`score`), puis mets a jour l'ecran (`afficher`). Dans cet ordre. Toujours.

Criteres de reussite

Les trois boutons marchent. Le score ne descend jamais sous zero. `const` pour les elements DOM, `let` pour `score` (car il change). Fonction `afficher` courte et reutilisee. Script avant `</body>`. Tu as clique partout sans erreur en console. C'est entre.



Exemple : chaque clic augmente le compteur.

Petite histoire

Lea chronometre parfois ses stagiaires : "version 1 en vingt minutes, sans perfection". Max a ajoute un son bete au clic +1, ses neveux ont crie comme au basket. Sam interdit le copier-coller sans retaper : "les doigts apprennent ce que les yeux survolent". Tu peux retaper ligne par ligne. C'est voulu. Le projet finit quand tu as teste chaque bouton dix fois et que tu peux expliquer chaque ligne a voix haute.

Erreur classique

Oublier `afficher()` apres un changement de score : la variable bouge en memoire, l'ecran reste fige. Laisser descendre sous zero : oublier le `if (score > 0)`. Selecteurs faux : `#score` vs `#Score`. Mettre `score` en `const` puis tenter de reassigner : erreur immediate. Autre piege : tout coller dans un seul listener geant. Decoupe. Un listener par bouton. Respire.

♦ ASTUCE

Si l'ecran ne bouge pas : loggue `score` dans le listener. Si le chiffre change en console mais pas a l'ecran, tu as oublie `afficher()`.

Bonus (quand la base marche)

Message si score ≥ 10 ("Bravo champion !"). Couleur differente via une classe CSS togglee. Bouton +5 pour accelerer. Plus tard : sauver dans `localStorage` (chapitre 18). Mais d'abord : la base qui marche. DanielCraft insiste : version 1 livrable avant version 2 fancy.

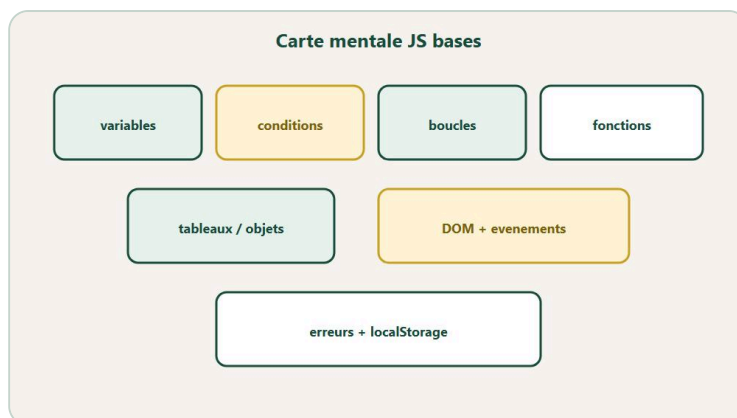
En vrai

Construis-le en une session courte. Une heure max si tu retapes. Clique partout. Casse volontairement un id, regarde l'erreur en console, repare. Tu valides la chaine complete : HTML, CSS, JS, DOM, evenements. C'est un vrai cran. Lea chronometre parfois : "version 1 en vingt minutes". L'objectif, c'est finir, pas peaufiner eternellement.

A toi

Ajoute une regle perso (exemple : score max 20, ou message a 10). Documente-la en trois lignes dans un commentaire ou un post-it. Chez DanielCraft, un projet fini avec une note vaut mieux qu'un projet "presque" jamais montre. Montre ton compteur a quelqu'un - meme trente secondes. Le geste compte.

Chapitre 14 - Ce qu'il faut retenir



Carte des bases : logique, données, DOM, mémoire.

Presque fini. Encore l'atelier todo, localStorage, la lecture des erreurs, le quiz, le bravo. Ce chapitre ne recrache pas tout le livre mot pour mot. Il range la boîte à outils dans ta tête pour que tu saches quoi rouvrir quand tu bloques demain matin sur un selecteur faux ou un listener silencieux. Retenir, chez DanielCraft, ce n'est pas décorer sa mémoire. C'est savoir où est le bon outil et comment l'utiliser sans panique.

JavaScript rend la page interactive. Tu ranges des infos avec **const** et **let**. Tu manipules texte, nombres, booleans. Tu decides avec **if**, tu repètes avec des boucles, tu reutilises avec des **fonctions**. Les tableaux sont des listes ordonnées. Les objets sont des fiches avec des propriétés nommées. Avec le **DOM**, tu trouves et tu modifies des éléments. Avec les **événements**, tu réagis - surtout au **click** pour commencer. Lea revoit encore ces bases avant des démos clients. Max est fier de son compteur et sait qu'il grandira. Sam dit à ses élèves : retenir, c'est refaire. Une checklist ne remplace pas le geste des doigts sur le clavier.

Imagine une ceinture à outils accrochée au mur. Dessus : fichier **.js**, variables, types, **if**, boucles, fonctions, tableaux, objets, **querySelector**, **textContent** / **classList**, **addEventListener**. Chaque outil a sa place. Les ateliers qui viennent (todo, localStorage) vont te forcer à t'en servir sous pression douce. Le quiz va secouer ce que tu crois savoir. Tu n'as pas besoin de tout tenir en mémoire en même temps. Tu as besoin de savoir où chercher et comment tester.

Ce que ce n'est pas

Ce n'est pas "tu maîtrises tout JavaScript". Loin de là. Ce n'est pas les frameworks React ou Vue - pas encore, pas ici. Ce n'est pas la fin du chemin : c'est un socle solide sur lequel tu peux construire. Lea revoit encore des bases avant des démos importantes. Max est fier de son compteur et sait qu'il y ajoutera localStorage puis autre chose. Sam dit : retenir, c'est refaire. Une checklist ne remplace pas le geste. Ne confonds pas "j'ai lu" et "je sais faire".

Habitudes solides

Script avant **</body>** pour que le HTML existe quand le JS tourne. Noms clairs : **scoreEl** plutôt que **s**. **console.log** pour vérifier avant de deviner. Petites fonctions courtes : **afficher()**, pas un bloc de cent lignes. **===** pour comparer, pas **==**. Tester après chaque petit changement, pas après avoir écrit cinquante lignes. Lire les erreurs en console au lieu de tout réécrire d'un coup. Une hypothèse, un test,

une correction. Chez DanielCraft, c'est la methode anti-drama. Lea la suit. Max l'a apprise a la dure. Sam l'enseigne des le premier cours.

★ A RETENIR

Declarer clair. Comparer avec `===`. Trouver avant de modifier. Ecouter avant d'agir. Lire la console.

Erreurs classiques a connaitre par coeur

Selecteur DOM faux : tu obtiens `null`, puis "Cannot read properties of null". Fonction ecrite mais jamais appelee : le code existe, rien ne se passe. Boucle `while` sans fin : la page freeze, tu fermes l'onglet. Script trop haut dans le HTML : les elements n'existent pas encore. `"5" + 2` donne `"52"`, pas `7` : types melanges. `=` dans un `if` au lieu de `===` : bug sournois. Oubli de `return` dans une fonction qui doit renvoyer quelque chose. Index de tableau : le premier element est `0`, pas `1`. Tu les croiseras. Tu les reconnaitras. Chaque reconnaissance te fait gagner dix minutes de frustration.

Petite histoire

Lea garde une checklist JS a cote de sa checklist HTML/CSS sur un post-it rose. Max a la version post-it jaune sur son ecran d'atelier. Sam fait recopier la checklist en fin de sequence : six lignes max, sans regarder le cours. Ceux qui oublient "script avant body" ou "=== pas ==" la recopient deux fois. Tu peux ecrire la tienne maintenant : six lignes max. Pas un roman. Un filet de securite pour les jours ou le cerveau est fatigue. Ce qui manque quand tu ecris sans regarder = ce qu'il faut revoir avant l'atelier todo.

Suite possible

Atelier todo : input, clic, creation d'elements. localStorage : se souvenir apres refresh. Mieux lire les erreurs : methode en cinq etapes. Puis formulaires plus riches, `fetch`, outils modernes. Mais la : bases d'abord. Elles portent tout le reste. Lea le repete aux clients impatientes de "passer a React". Max comprend maintenant. Sam le dessine au tableau : socle, puis etages.

⚠ ATTENTION

Un framework sur un sol mou, ca penche. Les bases de ce livre portent tout le reste. Ne saute pas a React avant de savoir selectionner un bouton.

En vrai

Rouvre ton compteur du chapitre 13. Ajoute un seul bonus sans relire le chapitre entier : un message si score ≥ 10 , ou un bouton +5. Si tu y arrives en moins de quinze minutes, c'est entre. Sinon, relis juste la piece qui manque - pas tout le livre. Cible. Teste. Corrige. C'est la methode DanielCraft : petit, clair, actionnable.

A toi

Ecris trois forces ("je sais brancher un clic"), deux faiblesses ("je confonds parfois const et let"), une prochaine etape (todo, localStorage, ou quiz). Date le papier si possible. Plan post-parcours. Style DanielCraft : petit, clair, honnete, actionnable. Tu sauras ou reprendre sans te noyer.

Chapitre 17 - Atelier : une todo liste mini



Saisir, ajouter, afficher, supprimer.

On passe a la pratique guidée. Tu vas construire une liste de taches : écrire une tache, cliquer pour l'ajouter, la voir apparaitre dans une liste. En bonus, la supprimer d'un clic ou valider avec la touche Entree. L'objectif n'est pas une app parfaite. C'est sentir le combo **input** + **clic** + creation d'elements **DOM** - le trio que tu retrouveras partout. Duree prevue : 30 a 45 minutes si tu avances pas a pas. Chez DanielCraft, la todo est l'atelier qui fait "waouh" parce que tu construis une mini appli visible, utilisable, montrable.

Lea en a livre des versions polish pour des clients qui voulaient "quelque chose d'interactif sans refaire tout le site". Max s'en sert pour ses courses du samedi - oui, vraiment, une todo perso. Sam chronometre le premier **li** qui apparait en classe : quand il tombe, il y a souvent des sourires. Trois metiers, meme logique : d'abord ca marche en memoire vive (DOM), ensuite ca se souvient (localStorage au chapitre suivant). Lea dit toujours : "geste d'abord, tiroir ensuite".

Tu ecris dans le champ. Tu cliques Ajouter. Un item naît dans la liste. Tu cliques dessus plus tard pour le retirer (bonus). La memoire vive, c'est le DOM. Le tiroir sur le disque du navigateur, ca viendra. La, on valide le geste de base. Sans geste solide, sauvegarder ne sert a rien.

Ce que ce n'est pas

Ce n'est pas encore localStorage - chapitre 18 juste apres. Ce n'est pas une app collaborative multi-utilisateurs. Ce n'est pas cent options (priorites, tags, dates). Ajouter proprement suffit pour valider l'atelier. Et ce n'est pas accepter le vide : une tache pleine d'espaces avec `trim()` oublie, ce n'est pas une tache. Sam refuse ca des la premiere version. Max a eu des "fantomes" dans sa liste avant d'apprendre.

HTML

```
<h1>Mes taches</h1>
<input id="champ" type="text" placeholder="Nouvelle tache">
<button id="ajouter">Ajouter</button>
<ul id="liste"></ul>
<script src="script.js"></script>
```

Quatre elements cles : le champ, le bouton, la liste vide qui va se remplir, le script en bas. Les ids doivent matcher ton JS. Une faute, silence total.

JS de base

```
const champ = document.querySelector("#champ");
const bouton = document.querySelector("#ajouter");
const liste = document.querySelector("#liste");

bouton.addEventListener("click", function () {
  const texte = champ.value.trim();
  if (texte === "") {
    return;
  }
  const li = document.createElement("li");
  li.textContent = texte;
  liste.appendChild(li);
  champ.value = "";
  champ.focus();
});
```

Pourquoi `trim()` ? Pour enlever les espaces avant et apres le texte saisi. Sinon tu ajoutes une tache "vide" pleine d'espaces invisibles. Moche. Frustrant. Sam refuse ca des la premiere version. Le `return` sur texte vide sort de la fonction sans rien creer : pas de fantome. Vider le champ et `focus()` remet le curseur pret pour la tache suivante. Petit detail, gros confort. Lea l'ajoute systematiquement.

♦ ASTUCE

Apres ajout : vide le champ et `focus()` . Le clavier reste pret. Tu enchaines les taches sans re cliquer dans le champ.

Bonus (quand la base marche)

Valider aussi avec Entree :

```
champ.addEventListener("keydown", function (event) {
  if (event.key === "Enter") {
    bouton.click();
  }
});
```

Supprimer au clic sur l'item :

```
li.addEventListener("click", function () {
  li.remove();
});
```

Dis-toi que cliquer sur une tache = "c'est fait, je retire". Simple. Clair. Suffisant pour l'atelier. Tu peux livrer sans les bonus. Avec, tu gagnes en fierte et en confort d'usage.

Petite histoire

Lea ajoute toujours `focus()` apres clear : ses clients enchainent les saisies sans friction. Max oubliait `trim` et avait des lignes vides fantomes dans sa liste de courses - sa femme a ri, puis a insiste pour le fix. Sam refuse les todos qui acceptent le vide : "si tu valides rien, rien ne doit naitre". Trois details, une appli plus humaine. Tu peux livrer la version de base. Avec les bonus, tu montres que tu penses utilisateur, pas

seulement code.

Erreur a eviter

Creer le `li` hors du listener : tu n'ajoutes qu'une fois au chargement. Oublier `appendChild` : element fantome invisible. Ne pas gerer le vide : liste pleine de fantomes. Tout coller sans tester item par item : tu ne sais pas ou ca casse. Vouloir localStorage avant que l'ajout marche : ordre DanielCraft, geste d'abord, memoire ensuite. Lea a vu des juniors bloquer une semaine sur la sauvegarde d'une todo qui n'ajoutait rien.

⚠ ATTENTION

Si rien n'apparait : loggue `texte` apres `trim`. Verifie `appendChild`. Verifie que tu ecoutes bien le bon bouton et le bon id.

Livrable

Une todo qui ajoute au minimum, avec `trim`, champ vide, et cinq lignes de lecons apprises (sur un post-it ou en commentaire). Bonus Enter + delete au clic. Montre-la a quelqu'un. Meme trente secondes. Le geste compte.

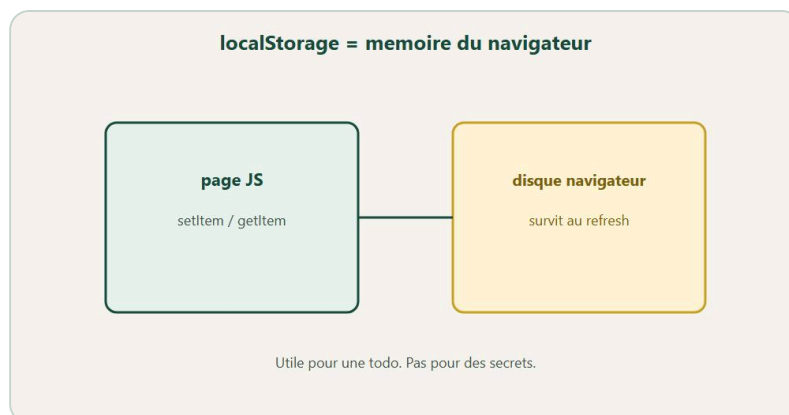
En vrai

Ajoute trois taches reelles de ta semaine. Utilise-la une journee entiere si tu peux. Note ce qui manque (sauvegarde apres refresh ? tri ?). Tu prepares localStorage consciemment. DanielCraft aime les outils qu'on ose utiliser soi-meme - pas les demos jamais rouvertes. Si ta todo disparaît au F5, c'est normal pour l'instant : le chapitre 18 est la pour ca.

A toi

Construis la version de base. Teste avec trois taches reelles. Ajoute au moins un bonus (Enter ou delete). Ecris cinq lignes "ce que j'ai appris". Puis passe au chapitre 18 si tu veux que ta liste survive au F5. Montre-la a quelqu'un. Meme trente secondes. Le geste compte autant que le code.

Chapitre 18 - Garder des infos (localStorage)



Sauvegarder dans le navigateur pour survivre au refresh.

Jusqu'ici, tout vivait en memoire vive. Tu fermes l'onglet ou tu appuies sur F5 : le compteur repart a zero, la todo s'evapore. Des fois, tu veux que ca reste. **localStorage** range des textes dans le navigateur, sur cet appareil, dans ce navigateur precis. Tu recharges la page : les donnees sont encore la. Petit pouvoir, gros effet "waouh" quand tu le montres a quelqu'un. Chez DanielCraft, on l'enseigne apres la todo et le compteur, parce que sans logique metier qui marche, sauvegarder ne sert a rien. Lea sauve des brouillons UI. Max sauve son score de basket salon. Sam montre aussi les limites avant la feerie : ce n'est pas un coffre-fort, ce n'est pas synchronise entre telephones.

Un tiroir dans ce navigateur precis, sur cet ordinateur. Tu y glisses des papiers texte etiquetes ("score", "taches", "prenom"). Tu peux les relire demain, apres un refresh, apres avoir ferme l'onglet. Si tu changes de navigateur, de machine, ou si tu vides les donnees du site, le tiroir est vide. Utile pour des preferences, un brouillon, un score perso. Pas pour des secrets. Lea explique toujours les limites avant la demo. Sam fait perdre les donnees volontairement en cours pour enseigner la fragilite.

```
localStorage.setItem("prenom", "Nora");
const prenom = localStorage.getItem("prenom");
console.log(prenom);
```

Tu ranges avec **setItem**. Tu relis avec **getItem**. Tu vis. Et tu te souviens que tout ca reste local a ce navigateur, sur cet appareil. Change de navigateur ou vide le cache : le tiroir est vide. Utile. Local. Fragile. Les deux faces du meme outil.

Ce que ce n'est pas

Ce n'est pas une base de donnees serveur. Rien n'est envoye sur internet automatiquement. Ce n'est pas synchronise entre appareils : le score sur le PC du bureau ne suit pas sur le telephone. Ce n'est pas safe pour mots de passe, tokens ou donnees sensibles : n'importe qui avec acces au navigateur peut lire localStorage. Ce n'est pas magique : ca stocke des **strings**. Pour nombres et listes, tu passes par conversion ou **JSON**. Et ce n'est pas "indestructible" : l'utilisateur peut vider le cache. Max a crie de joie, puis autrement, le jour du cache vide.

Nombres et tableaux

```
const score = 12;
localStorage.setItem("score", String(score));
const lu = Number(localStorage.getItem("score"));

const taches = ["acheter du pain", "reviser JS"];
localStorage.setItem("taches", JSON.stringify(taches));
const retrouvees = JSON.parse(localStorage.getItem("taches") || "[]");
```

Tout est string dans localStorage. Un nombre devient "12". Un tableau devient une longue chaîne JSON. `JSON.stringify` transforme en texte. `JSON.parse` reconstitue. Le `|| "[]"` évite de parser `null` quand rien n'a encore été sauve : petit garde-fou, gros calme. Lea met ce pattern partout. Max l'a appris après un crash au chargement.

⚠ ATTENTION

Tout est string dans localStorage. Nombres et tableaux : convertis. Et ne stocke jamais de secrets ici.

Brancher sur la todo ou le compteur

Pseudo-plan pour la todo : un tableau `taches = []` en mémoire ; à chaque ajout ou suppression : modifier le tableau, sauvegarder avec `JSON.stringify`, mettre à jour le rendu ; au démarrage : charger avec `JSON.parse`, puis afficher chaque tâche. Pour le score du chapitre 13 : `setItem` à chaque changement de score, `getItem` au chargement avant d'afficher. Ordre important : charger, afficher, puis écouter les clics. Sinon tu écoutes avec un score à zéro alors que le tiroir en contient un autre.

Au load : `score = Number(localStorage.getItem("score") || "0")` puis `afficher()`. À chaque clic qui change le score : modifier, afficher, `localStorage.setItem("score", String(score))`. Ce pattern tient pour presque tout ce que tu sauveras en débutant.

★ A RETENIR

Charger au démarrage, afficher, puis écouter. Sauver après chaque changement. Ordre : geste d'abord, tiroir ensuite.

Petite histoire

Max a crié quand son score a survécu au F5 pour la première fois. Il a montré l'écran à toute la famille. Puis il a vidé le cache du navigateur en voulant "nettoyer" et a crié autrement. Lea explique toujours les limites avant la magie : pouvoir + prudence. Sam fait perdre les données volontairement en fin de séance : "regardez, ce n'est pas magique". Trois scènes, une posture : utiliser l'outil en connaissance de cause.

Erreur classique

Oublier que tout est string : tu compares `"12" > 10` sans convertir, surprises possibles. Parser sans garde-fou : `JSON.parse(null)` plante. Stocker des secrets : mauvaise idée, point. Croire que ça marche sur un autre téléphone automatiquement : non. Sauvegarder sans recharger le rendu au démarrage : les données existent dans le tiroir, l'écran reste vide. Sauvegarder avant que l'ajout marche : ordre geste d'abord, mémoire ensuite. Lea insiste sur l'ordre load -> afficher -> écouter.

En vrai

Branche localStorage sur ton compteur du chapitre 13. Sauvegarde a chaque changement. Recharge la page : le score revient. Ouvre les outils developpeur (Application / Storage), vide le stockage du site, recharge : le score repart a zero. Tu as vu les deux faces. Puis pense a ta todo : meme logique avec un tableau JSON. Lea explique toujours les limites avant la feerie. Toi aussi, tu les as vues de tes yeux.

A toi

Branche localStorage sur ta todo : sauvegarde a chaque ajout et suppression, restauration au load. Ecris trois lignes sur ce que tu ne stockeras jamais ainsi (mots de passe, donnees clients, tokens). Posture DanielCraft : pouvoir + prudence. Montre la version qui survit au F5 a quelqu'un. L'effet "waouh" vaut le detour - a condition de connaitre la fragilite du tiroir.

Chapitre 19 - Erreurs JS frequentes (et comment les lire)



F12 : lire le message, la ligne, puis corriger.

Le navigateur parle. Apprends à l'écouter au lieu de paniquer. La **console** (F12, onglet Console) affiche du rouge quand quelque chose bloque. Souvent avec un numero de ligne et un fichier. Lire ça, ce n'est pas être nul. C'est avancer. Chez DanielCraft, on dit que debugger, c'est un métier - pas une punition. Lea lit les stacks tous les jours sur des projets clients. Max a gagné en calme le jour où il a suivi la ligne indiquée au lieu de tout effacer. Sam transforme chaque message d'erreur en énigme devant ses élèves : "qu'est-ce que le navigateur essaie de te dire ?"

Le navigateur est un collègue sec mais honnête. Il ne te jugera pas. Il te dit où ça fait mal : fichier, ligne, type d'erreur. Toi, tu vas à la ligne indiquée, tu poses un **console.log** juste avant pour voir ce qui se passe, tu corriges le plus petit truc possible, tu retestes. Max préfère ça aux soirées entières à tout recommencer. Sam affiche les messages au vidéo-projecteur comme des indices de jeu d'enquête. L'erreur devient un puzzle, pas une humiliation.

Tu vas croiser "X is not defined" : variable absente, faute de frappe, mauvaise portée. "Cannot read properties of null" : **querySelector** a renvoyé vide, script trop haut, id faux. "Unexpected token" : syntaxe cassée - parenthèse, crochet, guillemet oublié. Boucle infinie : page freeze, fan du PC qui tourne, **while** sans fin. Chaque message a une cause typique. Tu apprends le dialecte. Ensuite, tu paniques moins. Tu corriges plus vite. Lea raconte qu'un junior a réécrit 80 lignes pour une typo sur la ligne 12. Elle lui a appris à lire la ligne 12.

Ce que ce n'est pas

Ce n'est pas paniquer et tout effacer pour recommencer de zéro. Ce n'est pas ignorer le numero de ligne parce que "c'est du charabia". Ce n'est pas "ça marche sur mon PC" sans lire le message. Une méthode anti-stress bat dix réécritures aveugles. Et ce n'est pas une honte : Lea, Max et Sam cassent encore régulièrement. La différence, c'est la méthode. Lea appelle ça "chirurgie", pas "démolition".

Méthode en 5 étapes

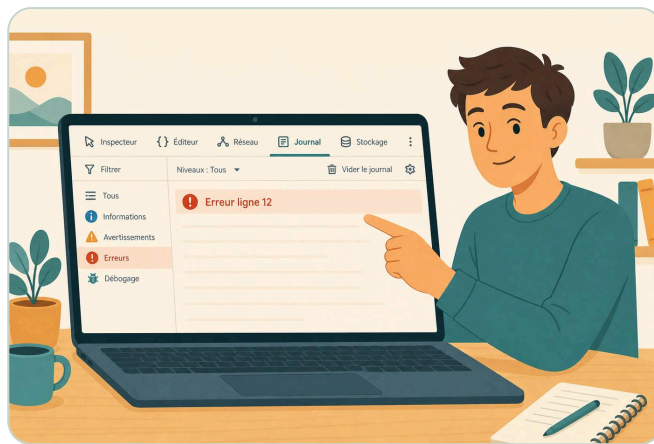
1. 1. Lis le message (même si tu ne comprends pas tout).
2. 2. Va à la ligne indiquée dans ton fichier.
3. 3. **console.log** juste avant pour voir l'état des variables.
4. 4. Corrige le plus petit truc possible (une typo, un id, une parenthèse).
5. 5. Reteste. Recommence si une nouvelle erreur apparaît.

★ A RETENIR

Lis. Va a la ligne. Log. Corrige petit. Reteste. Cinq gestes, moins de drama.

Erreurs frequentes detaillees

"X is not defined" : faute de frappe (`scoree` vs `score`), variable declaree dans un bloc mais utilisee ailleurs, script charge avant la definition. "Cannot read properties of null" : selecteur faux, `#id` qui n'existe pas, script place trop haut dans le HTML avant que l'element existe. "Unexpected token" : relis les guillemets, les accolades, les parentheses autour de la ligne indiquee - souvent une virgule ou un `)` manquant. Boucle infinie : la page freeze, coupe l'onglet, ajoute l'increment manquant dans ton `while`, reprends calmement. `"5" + 2` donne `"52"` : types melanges, pas une erreur rouge mais un bug logique. `=` dans un `if` au lieu de `===` : le code tourne, le resultat est faux.



Exemple : on lit l'erreur dans le journal, sans paniquer.

Petite histoire

Lea raconte un junior qui a reecrit 80 lignes pour une typo sur un nom de variable. Elle lui a appris a lire "line 12" et a corriger deux caracteres. Vingt secondes. Max a freeze son navigateur avec un `while` sans increment, a eu peur, puis a ri en voyant que c'etait juste une ligne oubliee. Sam affiche des messages d'erreur au video-projecteur : "Cannot read properties of null - qui veut deviner ?" Les eleves cherchent. L'erreur devient un jeu. Toi aussi, tu peux jouer.

◆ ASTUCE

Casse volontairement un truc qui marche. Renomme un id. Oublie une parenthese. Lis l'erreur. Repare. Tu entraines le reflexe sans la pression d'un vrai bug en production.

Exercice atelier

Casse volontairement ton compteur : renomme un id dans le HTML sans toucher au JS. Lis l'erreur. Repare. Casse une `const` en tentant de la reassigner. Lis. Repare. Casse une parenthese dans une fonction. Lis. Repare. Trois casses, trois lectures, trois victoires. Le calme s'installe avec la repetition. Pas magie. Habitude.

En vrai

Ouvre la console sur une page qui marche (ton compteur ou ta todo). Provocation volontaire : une typo dans un selecteur. Observe le message rouge. Va a la ligne. Corrige. Observe la disparition du rouge. Le reflexe s'installe. Prochaine panique réelle, tu commences par F12 au lieu de tout effacer. Lea appelle ca "chirurgie". Max prefere ca aux soirees a tout recommencer. Toi aussi, tu peux.

A toi

Cree une mini fiche "mes 4 erreurs" avec pour chacune : le message, la cause typique, le fix en une ligne. Colle-la pres de ton ecran. Prochaine panique, tu commences par la fiche, pas par le bouton "tout supprimer". Reflexe DanielCraft : methode avant drama. Dans une semaine, ajoute une cinquieme erreur que tu auras croisee. La fiche grandit avec toi.

QUIZ

Quiz final - Teste-toi !

Quiz : verifier, pas inventer

Lis la question

Choisis A / B / C

Si doute : reouvre le chapitre

Une mauvaise reponse = une piste de revision.

Verifier ce que tu as compris, sans inventer.

Meme regle que dans tout le parcours : cherche d'abord, corrige ensuite. Ce quiz n'est pas la pour te coller une note sur le front. C'est pour verifier que **JavaScript** n'est plus un brouillard flou dans ta tete : variables, types, conditions, boucles, fonctions, tableaux, DOM, evenements. Chez DanielCraft, un quiz sert surtout a cibler les chapitres a relire - pas a mesurer ta valeur humaine. Lea revoit parfois ces bases avant une demo importante. Max a refait le quiz un samedi matin et a relu le chapitre DOM. Sam l'utilise en fin de sequence pour voir ce qui reste fragile.

Lis chaque question calmement. Reponds dans ta tete ou sur papier. Puis descends aux corriges. Note tes hesitations : elles valent plus que les reponses faciles. Si tu bloques sur Q9 et Q11, tu sais ou aller (chapitres 10 et 12). Si tout passe, tu peux attaquer le bravo en confiance. Le score mesure un instant, pas ton potentiel.

Questions

Avant de commencer, respire. Douze questions, douze checkpoints. Pas de piege volontaire. Juste le socle que tu as construit chapitre apres chapitre.

Q1. JavaScript sert surtout a :

- A) Structurer la page
- B) Styliser la page
- C) Rendre la page interactive

Q2. Ou placer le `<script src="script.js">` pour debuter simplement ?

- A) Avant `</body>`
- B) Dans le CSS
- C) Apres `</html>`

Q3. Quelle declaration est faite pour ne pas etre reassignee ?

- A) `let`
- B) `const`
- C) `var` (et on s'en fiche pour l'instant)

Q4. `true` et `false` sont des :

- A) strings
- B) booleans
- C) tableaux

Q5. Quelle comparaison est recommandee ici ?

- A) `==`
- B) `===`
- C) `====`

Les cinq premieres couvrent le socle : role de JS, placement du script, variables, types, comparaisons. Si tu hesites ici, revois les chapitres 1 a 5 avant de continuer.

Q6. Une boucle `for` sert a :

- A) Stocker une image
- B) Repeter une action
- C) Changer la police

Q7. `return` dans une fonction :

- A) Affiche toujours dans la page
- B) Renvoie une valeur
- C) Efface la variable

Q8. Dans `["a", "b"]`, l'index de `"a"` est :

- A) 1
- B) 0
- C) 2

Q9. `document.querySelector("#titre")` selectionne :

- A) Une classe `titre`
- B) Un id `titre`
- C) Toutes les balises titre

Q10. `textContent` sert a :

- A) Changer le texte d'un element
- B) Creer un serveur
- C) Compresser une image

La moitie du quiz. Fonctions, tableaux, DOM. C'est la que Lea revoit le plus souvent avant de livrer une demo interactive.

Q11. `addEventListener("click", ...)` :

- A) Change la couleur tout seul
- B) Ecoute un clic pour lancer du code
- C) Installe JavaScript

Q12. `event.preventDefault()` sur un submit :

- A) Envoie le formulaire plus vite
- B) Empêche le comportement par défaut
- C) Supprime le bouton

Les deux dernières questions vérifient que tu as le réflexe interaction : écouter, intercepter. Si tu as hésité sur Q11 ou Q12, le chapitre 12 mérite dix minutes de relecture ciblée.

Reponses

1. **C** - JavaScript rend la page interactive. HTML structure, CSS stylise.
2. **A** - Avant `</body>`, le HTML existe quand le script tourne.
3. **B** - `const` pour ce qui ne change pas. `let` si tu réassignes.
4. **B** - Booleans pour vrai/faux, conditions, comparaisons.
5. **B** - `===` compare valeur et type. Pas `==`, pas `====`.
6. **B** - Répéter une action un nombre de fois ou sur une liste.
7. **B** - `return` renvoie une valeur à l'appelant. Pas d'affichage automatique.
8. **B** - Index 0 pour le premier élément. Classique piège débutant.
9. **B** - `#` cible un id. `.` ciblerait une classe.
10. **A** - Changer le texte visible d'un élément DOM.
11. **B** - Écouter un événement pour exécuter du code au bon moment.
12. **B** - Empêche le reload du formulaire pendant tes tests.

Score

- 10-12 : propre, le socle tient. Passe au bravo et montre ton compteur à quelqu'un.
- 7-9 : bien, relis DOM + événements (chapitres 10, 11, 12) puis reteste les questions floues.
- moins de 7 : refais les chapitres 3, 5, 10, 12 avec les "A toi" puis reviens au quiz.

★ A RETENIR

Note tes 3 questions les plus floues. Cinq minutes de relecture par chapitre lié. Puis reteste ces 3 seulement, sans regarder les corrigés.

Petite histoire

Sam rappelle toujours : le score mesure un instant, pas ta valeur. Lea a raté des QCM en formation et code proprement aujourd'hui sur des projets clients. Max a eu 8 au premier passage, a relu le chapitre DOM en une pause café, a refait le quiz à 11. Il a souri. Toi aussi, tu peux. Le chiffre est une carte, pas un jugement. Ce qui compte, c'est ce que tu sais refaire avec les doigts.

En vrai

Sans regarder les corrigés, liste tes 3 questions les plus floues. Cinq minutes de relecture par chapitre lié. Puis reteste ces 3 seulement. L'important n'est pas d'être parfait du premier coup. C'est de progresser à chaque passage. Ensuite, lis le chapitre bravo. Tu l'as mérité. Lea revoit parfois ces bases avant une démo. Toi aussi, tu peux revenir sans honte.

A toi

Note ton score, tes 3 hésitations, et la date. Dans une semaine, reteste les questions ou tu as doute. Si tu passes de 7 à 11, tu verras la progression concrète. Style DanielCraft : petit, mesurable, honnête. Le chiffre est une carte, pas un jugement sur ta valeur.

Bravo.



Bravo. Tu as appris à rendre une page vivante.

Tu viens de terminer les bases de **JavaScript**. Pas "je connais tout le web du monde entier". Mais "je sais faire réagir une page avec mes propres scripts". C'est déjà un vrai cran. Chez DanielCraft, on célèbre ce passage parce qu'il est rare : beaucoup de gens lisent des tutos, peu finissent un compteur qui marche, une todo utilisable, une erreur lue sans abandonner. Toi, tu as tenu le fil des variables jusqu'aux événements. Tu as debuggé. Tu as peut-être sauvé dans localStorage. Sérieux : bravo.

Lea se souvient encore de son premier bouton qui répondait à un clic - elle avait ouvert la console dix fois ce jour-là pour vérifier. Max a montré son score persistant à sa famille un dimanche soir ; ses neveux ont cliqué jusqu'à ce que le score soit bloqué à 20. Sam applaudit ceux qui ont lu une erreur rouge sans fermer l'onglet en rage. Tu es dans cette ligue maintenant. Le web commence à t'obéir. Doucement. Proprement. Et c'est exactement comme ça qu'on apprend.

Tu as donné un cerveau minimal à ta page. Elle écoute des clics. Elle répond avec du texte qui change. Elle peut même se souvenir un peu avec **localStorage** - score, tâches, préférences locales. Demain, elle pourra parler à des serveurs avec **fetch**. Aujourd'hui, elle t'obéit déjà quand tu cliques. Doucement. Proprement. C'est le cran qui change tout : passer de "je lis du code" à "je fais du code qui marche".

★ A RETENIR

Tu sais faire réagir une page. C'est le cran qui change tout. Garde le geste : petit, clair, testable.

Ce que ce n'est pas

Ce n'est pas la fin du chemin JavaScript. Loin de là. Ce n'est pas un framework maîtrisé. Ce n'est pas l'obligation d'enchaîner **fetch** et les API ce soir. Si tu es fatigué, repose-toi. Tu l'as mérité. Demain, tu pourras monter. Aujourd'hui, tu souffles et tu regardes ce que tu as construit : un compteur, peut-être une todo, des lignes de JS que tu comprends.

Ce que tu sais faire maintenant

Ecrire un script et le placer au bon endroit. Ranger des infos dans des variables avec `const` et `let`. Decider avec `if`, repeter avec des boucles, reutiliser avec des fonctions. Manipuler des listes (tableaux) et des fiches (objets). Trouver et modifier le DOM avec `querySelector`, `textContent`, `classList`. Reagir aux clics avec `addEventListener`. Construire un compteur interactif. Creer une todo qui ajoute des taches. Sauvegarder localement avec `localStorage`. Lire une erreur en console au lieu de paniquer. Ce n'est plus du cours abstrait : ce sont des gestes que tu as faits.

Petite mission

Reprends ton compteur du chapitre 13. Ajoute un message si score ≥ 10 . Une couleur differente via une classe CSS. Un bouton +5 si tu veux aller plus loin. Montre-le a quelqu'un - meme trente secondes, meme en visio. L'important, c'est le geste de montrer. Lea envoie parfois ses premiers scripts aux stagiaires : "regarde d'ou je viens, pas ou je suis". Max garde une capture de son premier compteur. Sam dit que montrer solidifie le courage face aux prochaines erreurs - car il y en aura. Heureusement.

Petite histoire

Max a garde une capture d'ecran de son compteur a score 12 sur son telephone. Il la montre encore quand un ami dit "le code c'est pas pour moi". Sam dit que le bravo n'est pas une formalite : il solidifie le courage face aux prochaines erreurs. Chaque erreur lue calmement est une competence. Chaque projet fini est une preuve que tu peux. Lea rappelle : le premier script moche qui marche bat le dixieme tuto jamais ouvert.

La suite quand tu seras pret

Formulaires plus riches avec validation poussee. `localStorage` plus serre sur ta todo complete. Puis `fetch` pour parler a des API. Peut-etre un petit projet perso publie en ligne. Mais la, respire. Chez DanielCraft, on forme des gens qui livrent petit, souvent, proprement - pas des collectionneurs de tutos oublies dans dix onglets. Tu as le socle. Le reste s'empile dessus.

En vrai

Ouvre ton compteur ou ta todo. Clique. Recharge. Verifie que tout tient. Si `localStorage` est branche, vide le cache une fois pour voir la difference. Tu as vu les deux faces. C'est une competence, pas une anecdote. Puis respire. Tu as tenu le fil des variables jusqu'aux evenements. Chez DanielCraft, on celebre ce passage parce qu'il est rare.

A toi

Ecris trois lignes sur un papier ou un fichier texte : (1) une fierte concrete ("mon compteur marche"), (2) un chapitre a revoir si tu l'as identifie au quiz, (3) une date pour ta prochaine session code - meme vingt minutes. Encore bravo. Le web commence a t'obeir. Doucement. Proprement. Garde ce rythme. Montre ton projet a quelqu'un si tu peux : le geste de montrer solidifie autant que le geste de coder.

Tu as les briques. Maintenant, construis.